LoCoBench: A Benchmark for Logical-Coherence Scorers

Phase 1 — facets, error taxonomy, generation prompts, gold schema and metrics

FactReasoner — Ideation

August 2, 2026

Abstract

The logical-coherence pipeline mines thirteen Level-2 discourse senses between the atoms of a response, compiles them to five Level-1 Markov-network couplings, and reads a Logical Coherence Score (LCS) off the resulting graph. None of it is evaluated on real data: what exists today is nine hand-authored fixtures and one exact-value regression oracle. This document designs the benchmark that would fix that. It (i) takes the construction methodology of LoReFact [LoReFact] — the first long-form fact-checking dataset with logical annotations — and identifies precisely what its four-relation inventory can and cannot express about ours, finding that *three of our five factor tables have no existing gold data anywhere*; (ii) defines LoCoBench's *facet* system — the structural axes a datapoint is graded on, kept deliberately disjoint from the 36 subject *topics* it inherits wholesale from LoReFact and must cover in full — together with target counts and a deliberate departure from LoReFact's 78%-false corpus balance, which a graded score cannot use; (iii) defines a ten-operator perturbation taxonomy with, for each operator, a *per-readout* expected direction, including two strong falsifiable invariance tests that the taxonomy's own structure licenses; (iv) gives the complete generation and validation prompt suite, copy-ready, extending LoReFact's Tables 11–14 and 20–25 and replacing its prose "logical statement" stage with a machine-readable relation plan; (v) specifies the gold schema as a strict superset of the shipped fixture format; and (vi) defines the metrics for both gradable targets, with LoReFact-comparable analogues. Scoring is scoped to three readouts — `mean_marginal`, `consistency` and `log_partition`, one each for belief, event activity and mass — for the reasons given in Section 5.1. Two findings shape the design and are stated up front, because a benchmark that ignored either would silently test the wrong thing: the readouts are not jointly monotone under increasing coherence — one of them provably inverts at the concession rung — and the taxonomy's only fixed strength prior is unreachable on the production code path.

Contents

1 Motivation, and what LoReFact does and does not give us

1.1 The evaluation gap

The shipped coherence stack (`src/fact_reasoner/lcs/`) has a relation miner, a two-level taxonomy, a Markov-network builder shared with the factuality assessor, and four score readouts. Its empirical basis is:

- nine hand-authored fixtures in `data/lcs/*.json`, carrying atoms and prose `notes` but *no machine-readable gold relations*;
- one regression oracle — the AeroParts model, whose exact values (`mean_marginal` $= 0.587$, $\log Z = -9.75$, `consistency` $= 0.813$, `log_partition` $= 0.7831$) are pinned by a test;
- an evaluation *plan* (`research_plan.md` §9) naming perturbation ladders, Spearman-versus-human correlation, and three baselines [DiscoScore, G-Eval, SelfCheckGPT] — but no dataset, no facet inventory, and no gold format.

So we cannot currently answer the two questions that matter: *does the miner recover the relations a response actually draws?* and *does the LCS order responses by coherence?* This document designs the data that would answer both.

1.2 LoReFact

LoReFact [LoReFact] is the closest prior art and the right starting point. It argues that the dominant decompose-then-verify fact-checking paradigm verifies atomic claims individually and overlooks the *logical dependencies* among them, so a text that combines true facts through a false dependency is misjudged as factual. Its preliminary study found that SAFE, VeriScore and FactCheck-GPT omit the logical dimension on 60%, 78% and 48% of logic-related samples respectively.

Its dataset contributions, which we reuse:

- **A controlled three-stage generation pipeline**: for each topic, generate a logic-eliciting question; generate 20 atomic claims (10 correct, 10 incorrect); compose at least two *logical statements* from them; then draft a response that naturally embeds all of the statements. Full prompts in their Tables 11–14.
- **Function-call-style annotation prompts** (their Tables 20–25) for decomposition, checkworthiness filtering, logic-type detection and verification — a house style we adopt for our validation prompts in Section 6.
- **Scale and coverage**: 1,080 samples, 5,706 annotated logical statements, 36 topics across 4 domains, average response length 819 words, 3–7 logical statements per sample, inter-annotator agreement 92.18%.

1.3 Why extending it is not incremental

LoReFact is built for a different task, and the mismatch is structural (Table 1).

The consequence, developed in Section 3, is that LoReFact's label set reaches 7 of our 13 senses and 2 of our 5 couplings. EQUIVALENCE, EXCLUSIVE and CO_NECESSITY — three of the five pairwise factor tables the network can build — have **zero** gold data in any existing dataset. That is the strongest single argument for building this benchmark rather than annotating on top of theirs.

	LoReFact	**LoCoBench (this design)**
Relation inventory	**4** types: causal, conditional, temporal, concessive (+ `no_logical_relationship`)	**13** Level-2 senses compiling to **5** Level-1 couplings (+ NONE)
Relation label	binary logic-factuality per statement	graded $p = P(\tau \mid a_i, a_j) \cdot P(a_j \mid a_i, \tau)$
Score granularity	per-statement, arithmetic mean	**graph-level** readout off a Markov network
Direction	not distinguished	`directed` per sense; Cause-Effect vs. Effect-Cause
Exhaustiveness	not represented	CONTRADICTION vs. EXCLUSIVE turns on it
Resolution	not represented	Concession carries a resolver and a discount
Corpus balance	**78% logically false by design** (4,429 / 5,706)	must also be correct at the *coherent* end

Table 1: The gap between LoReFact's design and what a logical-coherence scorer needs. Each row is a dimension along which their gold labels cannot grade our system.

2 Two findings that shape the design

Both were verified against the source while designing, and both are stated before the design because a benchmark that assumed otherwise would silently measure the wrong thing.

2.1 Finding 1: the readouts are not jointly monotone — by construction

`research_plan.md` §9 states the natural requirement: "a good LCS must drop monotonically with perturbation severity." Applied to every readout alike, *this requirement is false*, and a benchmark enforcing it would fail a correct implementation.

The MRF deep-dive (§"The shared concession quirk") establishes why. The concession edit *softens* a resolved conflict from $p = 0.80$ to $p = 0.55$. A weaker EXCLUSIVE (or CONTRADICTION) factor puts *more* mass on the both-true cell — its $(1, 1)$ entry is $1 - p$ — so for a lone edge under uniform priors

$$P(a_{11}{=}1, a_{12}{=}1) \;=\; 0.10 \;\; (p{=}0.80) \qquad \longrightarrow \qquad 0.225 \;\; (p{=}0.55).$$

The readout that asks *"is the conflict active"* — `consistency`, which counts the event — therefore sees the softened edge as *more* often active and **dips**. Readouts that ask *"how strongly do we believe each atom"* — `mean_marginal` on the marginals, `log_partition` on the mass — correctly read the softening as *less* suppression and **rise**.

Two design consequences, carried through Sections 4, 5 and 8:

(1) Ordering contracts are **per readout**, not global (the C1/C2/C3 classes of Section 5.3).

(2) The inversion becomes a **positive prediction** rather than an exemption: a system whose `consistency` is monotone across the concession rung is *not* implementing the concession discount. Section 8.3 makes this the *quirk-conformity* metric, whose target is high.

2.2 Finding 2: Restatement's strength prior is unreachable in production

`COMPILE[RESTATEMENT].strength_prior = 0.90` is the taxonomy's only fixed prior, and `compile_sense` does use it — but only when its `raw_p` argument is `None`. The single call site in the miner is

```
compiled_level1, _, spec = compile_sense(sense, None)
```

which passes `None` *and discards the effective-strength return value entirely* (the `_`). Strength on the production path comes from Prompt B alone. The 0.90 prior therefore never reaches a factor table.

So the benchmark *cannot* test the prior through the pipeline. What it can test is whether the prior is *empirically justified*: whether mined strength on gold Restatement edges concentrates near 0.90. Section 9 records this as an open item (R4) rather than a metric, so no one later writes an assertion that silently passes.

3 The LoCoBench facet system

3.1 The facets of LoCoBench

Terminology, and a collision to avoid. LoReFact organizes its corpus by *subject matter*: 36 **topics** (Literature, Astronomy, Civil Engineering, ...) grouped into 4 **domains** (Table 6). LoCoBench inherits that grid wholesale — every one of the 36 topics must be represented (Section 3.2) — so those two words are **reserved throughout this document for subject matter, and never used for anything else**.

What LoCoBench adds is a second, orthogonal organization: the structural properties a datapoint is labelled with and graded on. We call these **facets**. A *facet* is one controlled vocabulary; a *facet value* is a value drawn from it. The two organizations are independent by construction — a single item has a topic *and* a full set of facet values, and the benchmark's coverage requirements apply to each separately.

	Subject organization (inherited)	**Facet system** (LoCoBench's own)
Terms	*topic*, *domain*	*facet*, *facet value*
Cardinality	36 topics in 4 domains	3 facet groups, 19 facets
Answers	"what is this text *about*?"	"what is being *tested* here?"
Source	LoReFact Table 9, adopted unchanged	new; derived from `lcs/taxonomy.py` and this design
Requirement	all 36 topics represented (§3.2)	per-facet targets (Tables 3–5)

Table 2: The two orthogonal organizations of the corpus. Keeping the vocabularies disjoint matters because both are coverage requirements and both are reported: a result broken down by *topic* says whether the benchmark generalizes across subject matter, while a result broken down by *facet* says which structural phenomena a scorer handles. Conflating them would make neither breakdown interpretable.

There are three groups of facets, listed here in one place so the rest of the document can refer to them without re-enumerating:

(1) **Relation facets** — the Level-2 *senses* and the Level-1 *couplings* they compile to (Table 3). These are what a gold edge is labelled with, and what the miner is graded on.

(2) **Item facets** — the axes along which items and families vary: family type, rung, perturbation operator, ordering contract, split (Table 4).
(3) **Annotation facets** — the enumerated values the schema's own fields take (Table 5). Each is defined operationally in a prompt or a schema note; that table is the index to them.

Within the relation facets, *sense* and *coupling* always name the two levels specifically, and are never used interchangeably with each other or with "facet".

(1) Relation facets. Thirteen senses compiling to five couplings plus NONE. The **coverage** column is the distinction that motivates the benchmark: which of these relations already have gold data somewhere, and which do not.

Level-2 sense	Level-1 coupling	Coverage	Role in the benchmark
Positive couplings — the relation supports its target			
Cause-Effect	ENTAILMENT	covered	the modal relation of explanatory prose
Effect-Cause	ENTAILMENT	covered	direction partner of Cause-Effect
Evidence	ENTAILMENT	partial	collapsed into "causal" by LoReFact
Condition	ENTAILMENT	covered	confusable with Precedence
Instantiation	ENTAILMENT	**new**	general↔specific; no prior gold
Restatement	EQUIVALENCE	**new coupling**	the only sense with a fixed prior (0.90)
Conflict couplings — define the "active conflict" event			
Contrast	CONTRADICTION	**new**	non-exhaustive opposition
Concession	CONTRADICTION	**new**	a tension the text resolves; 0.45 discount
Alternative	EXCLUSIVE	**new coupling**	*exhaustive* opposition: exactly one holds
Disjunctive coupling — not a conflict			
Disjunction	CO_NECESSITY	**new coupling**	at least one holds; penalizes only $(0, 0)$
No coupling — no factor is added to the network			
Precedence	NONE	none	ordering only; invariance target
Succession	NONE	none	ordering only; invariance target
None	NONE	none	the distractor label; over-connection denominator

Table 3: **The 13 relation facets of LoCoBench**, grouped by what their coupling does to the Markov network. new no prior gold data anywhere partial covered, but only coarsely none no MRF factor (NONE). Six senses are new: Instantiation, Restatement, Contrast, Concession, Alternative, Disjunction — and three of the five couplings (EQUIVALENCE, EXCLUSIVE, CO_NECESSITY) have *no gold data in any existing dataset*, which is the case for building this benchmark (Section 3.3). Property flags (directed, concession, ordering-only) and the normative compilation map are in Table 8; per-sense target counts in Table 9.

(2) Item facets. The axes along which items and families vary. Family type and rung together determine what a given item is *for*; the operator says how it was derived from its parent rung.

(3) Annotation facets. These are the enumerated values the schema's fields take (Section 7). Each is defined where it is operationally used — in a prompt's instructions, or a schema field note — so this table is the index rather than the definition.

Axis	#	Values
Family type	5	F-CONF, F-CHAIN, F-EXH, F-ORDER, F-INV (Table 14)
Rung	5	`worse`, `base`, `concession_resolved`, `fix_one_conflict`, `coherent` (Table 13)
Perturbation op.	10	P-WS, P-DR, P-EF, P-CR, P-IC, P-BC, P-OS, P-SR, P-DL, P-OO (Table 11)
Ordering contract	3	C1 full chain, C2 quirk chain, C3 endpoints (Section 5.3)
Confusable pair	11	weighted 1–3 (Table 10)
Readout	3	`mean_marginal`, `consistency`, `log_partition` (Section 5.1)
Split	2	`dev` (the 9 shipped fixtures), `test` (generated) (Section 5.5)

Table 4: **Item facets**: the axes along which benchmark items and families vary. Each row cites the section that defines it normatively. Subject matter is deliberately *not* listed here — topic and domain are the inherited organization of Section 3.2, orthogonal to every facet above (Table 2).

Field	#	Values	Defined in
`validity`	2	`valid`, `invalid`	P3, instr. 8
`error_kind`	4	`wrong_sense`, `wrong_direction`, `false_endpoint`, `spurious`	P3, instr. 8
`strength_band`	3	`strong` $[0.85, 1.00]$, `moderate` $[0.60, 0.84]$, `weak` $[0.35, 0.59]$	P3, instr. 7
`role` (atom)	3	`claim`, `holding`, `distractor`	§7
`window_admission`	4	`window`, `gate`, `discourse_promoted`, `out_of_window`	§7
`readout_directions`	4	`decrease`, `increase`, `invariant`, `unconstrained`	§7, Table 11
Claim tag (P2)	9	`[correct]`, `[incorrect]`, `[alt-pair-1/2]`, `[disj-pair-1/2]`, `[equiv-pair-1/2]`, `[holding]`	P2, instr. 7
V2 verdict	4	A exhaustive, B non-exhaustive, C disjunction, D neither	V2
V4 status	4	`asserted`, `altered`, `missing`, `merged`	V4

Table 5: **Annotation facets.** Two of these are load-bearing outside their defining prompt and are easy to miss: `error_kind` is a required gold field, and the `strength_band` ranges are what metric A7 (Section 8.1) grades against. Prompt references are to Appendix A.

3.2 Subject coverage: all 36 LoReFact topics

Requirement. LoCoBench must contain datapoints from **every one of the 36 topics** of LoReFact Table 9, reproduced here as Table 6. This is a hard coverage constraint, not a sampling preference: it is what makes a LoCoBench result comparable to a LoReFact result on the same subject matter, and what prevents the facet targets of Section 3.1 from being met inside a narrow slice of subject matter that happens to suit them.

Allocation. With 120 families over 36 topics, the target is $\lfloor 120/36 \rfloor = 3$ families per topic plus 12 families distributed to the topics that best support the scarce relation facets. Section 6 biases that surplus toward adjudicated subject matter — incident investigation, litigation, diagnostic reasoning — which is where exhaustive alternatives and resolving holdings occur naturally. Concretely the surplus goes to Law, Medicine, Civil Engineering, Political Science and Ethics, while the **3-family floor holds for all 36 topics without exception**.

Reporting. Every headline metric of Section 8 is reported both pooled and broken down by domain, as LoReFact does (their Table 3). A per-*topic* breakdown at 3 families per topic is too thin for a stable estimate, so topic coverage is verified as a *constraint* on the corpus rather than reported as 36 separate scores.

3.3 Mapping LoReFact's inventory onto ours

Table 7 maps each LoReFact type onto the senses it covers.

Natural Sci. (11)	**Humanities** (10)	**Engineering & Tech.** (8)	**Social Sci.** (7)
Astronomy	Archaeology	Artificial Intelligence	Anthropology
Biology	Art	Civil Engineering	Economics
Botany	Culture	Computer Science	Education
Chemistry	Ethics	Cybersecurity	Law
Geology	Gender Studies	Electrical Engineering	Political Science
Human Biology	History	Mechanical Engineering	Psychology
Medicine	Linguistics	Renewable Energy Tech.	Sociology
Meteorology	Literature	Robotics	
Oceanography	Philosophy		
Physics	Religion		
Zoology			

Table 6: **The 36 subject topics**, adopted unchanged from LoReFact Table 9 and grouped by their 4 domains. Every topic must contribute at least one family. Note that LoReFact's own example tables (their Tables 16–19) name five of these differently — *Animals*, *Human Body*, *Ocean*, *Plants*, *Weather* for *Zoology*, *Human Biology*, *Oceanography*, *Botany*, *Meteorology* — and two topics appear there with a duplicated example question. We treat Table 9 as authoritative and record the variance so a Phase-2 topic list can be diffed against either.

LoReFact type	**Covers our senses**	n	**Mismatch**
causal	Cause-Effect, Effect-Cause, Evidence	3,358	direction not distinguished; Evidence (epistemic) collapsed into causal (ontic)
conditional	Condition	1,551	1:1
temporal	Precedence, Succession	473	**semantic mismatch**: treated as truth-bearing, whereas we compile both to NONE (`ordering_only`)
concessive	Concession	324	no resolved/unresolved distinction; conflated with Contrast
`no_logical_rel.`	None	—	1:1
Senses LoReFact *structurally cannot express — no gold data exists for these*			
(none)	Alternative → EXCLUSIVE	0	no type encodes "exactly one holds" — the coupling penalizing *both* same-value worlds has no analogue in any prior dataset
(none)	Disjunction → CO_NECESSITY	0	their label set has no positive-polarity multi-claim coupling at all
(none)	Restatement → EQUIVALENCE	0	their claim prompt requires claims be "basic, indivisible, and independently verifiable", which *forbids* the paraphrase pairs an equivalence edge needs
(none)	Instantiation	0	general↔specific, silently absorbed into causal
(none)	Contrast (vs. Concession)	0	their single "concessive" type conflates unresolved non-exhaustive opposition with a resolved tension
(none)	Concession *with resolution*	0	the `is_concession` + resolver structure driving the $p_{\text{eff}} = p\,(1 - 0.45)$ discount is unrepresented, so the discount is untestable against their data

Table 7: LoReFact → LoCoBench sense coverage, with their statement counts (their Table 4). **Upper block:** the seven of our thirteen senses their four types reach, and how coarsely. **Lower block, shaded:** the six they cannot express, with a statement count of zero — these are why EQUIVALENCE, EXCLUSIVE and CO_NECESSITY have no gold data anywhere. new no prior gold data anywhere partial covered, but only coarsely none no MRF factor (NONE).

3.4 The normative compilation matrix

Table 8 reproduces `COMPILE` from `src/fact_reasoner/lcs/taxonomy.py`. **This table is normative**: a benchmark item whose `level1_coupling` disagrees with `COMPILE[level2_sense].level1` is malformed, and Section 7 requires a builder assertion to that effect.

Level-2 sense	Level-1 coupling	prior	directed	concession	ordering-only
Cause-Effect	ENTAILMENT	—	yes	—	—
Effect-Cause	ENTAILMENT	—	yes	—	—
Evidence	ENTAILMENT	—	yes	—	—
Condition	ENTAILMENT	—	yes	—	—
Instantiation	ENTAILMENT	—	yes	—	—
Restatement	EQUIVALENCE	0.90	no	—	—
Contrast	CONTRADICTION	—	no	—	—
Concession	CONTRADICTION	—	yes	**yes**	—
Alternative	EXCLUSIVE	—	no	—	—
Disjunction	CO_NECESSITY	—	no	—	—
Precedence	NONE	—	yes	—	**yes**
Succession	NONE	—	yes	—	**yes**
None	NONE	—	no	—	—

Table 8: The Level-2 → Level-1 compilation map (deep-dive Table 2), verbatim from `taxonomy.py`. Shading repeats the coverage classes of Table 3 (new no prior gold data anywhere partial covered, but only coarsely none no MRF factor (NONE)) so the property matrix and the inventory can be read together. Restatement is the only sense with a fixed strength prior — and see Finding 2 (Section 2.2) for why that prior is not reachable in production. Conflict couplings, which define the "active conflict" event for `consistency`, are exactly {CONTRADICTION, EXCLUSIVE}; CO_NECESSITY is a *positive* coupling and is not a conflict.

Two boundaries in this table carry the design's difficulty, and both need explicit annotation to be gradeable at all:

Contrast vs. Alternative — non-exhaustive versus exhaustive opposition. Contrast forbids only the both-true world; Alternative forbids both-true *and* both-false. This is the sole distinction in the taxonomy that moves a relation between CONTRADICTION and EXCLUSIVE, i.e. between two different factor tables. Deciding it requires quantifying over possible worlds ("could both be false?"), which is why Section 6 gives it a dedicated adjudication prompt with an explicit decision procedure, and Section 9 rates it our highest risk.

Contrast vs. Concession — unresolved versus resolved. Same coupling, but `is_concession` flips and a resolver atom must be identified, which changes the effective strength by the 0.45 discount.

3.5 Target counts and the composition of the corpus

Phase-2 target: **600 items** in **120 families** of 5 rungs (Section 5), spanning all 36 subject topics (Section 3.2) so cross-dataset comparison stays possible. At ∼8 gold edges per item that is ∼4,800 gold relations, comparable to LoReFact's 5,706 logical statements. Table 9 gives the per-sense allocation — the *facet* axis, orthogonal to the topic axis above.

Sense	Coupling	Edges	Share	Cov.	Rationale
Cause-Effect	ENTAILMENT	700	14.6%	covered	most frequent in natural discourse
Effect-Cause	ENTAILMENT	450	9.4%	covered	direction-confusion partner of the above
Evidence	ENTAILMENT	400	8.3%	partial	epistemic/ontic boundary with causal
Condition	ENTAILMENT	450	9.4%	covered	confusable with Precedence (LoReFact's own failure mode)
Instantiation	ENTAILMENT	350	7.3%	**new**	no prior gold
Restatement	EQUIVALENCE	350	7.3%	**new**	new coupling; no prior gold
Contrast	CONTRADICTION	400	8.3%	**new**	as distinct from Concession
Concession *(unresolved)*	CONTRADICTION	250	5.2%	**new**	the undiscounted baseline for the resolved case
Concession *(resolved)*	CONTRADICTION	250	5.2%	**new**	exercises the 0.45 discount and the resolver heuristic
Alternative	EXCLUSIVE	400	8.3%	**new**	new coupling; no prior gold
Disjunction	CO_NECESSITY	350	7.3%	**new**	new coupling; no prior gold
Precedence	NONE	200	4.2%	none	ordering-only invariance targets
Succession	NONE	200	4.2%	none	ordering-only invariance targets
None *(distractors)*	NONE	≥1,500	—	none	separate in-window pool; the denominator for over-connection

Table 9: Per-sense gold-edge targets. new no prior gold data anywhere partial covered, but only coarsely none no MRF factor (NONE). The **six** new senses — Instantiation, Restatement, Contrast, Concession, Alternative, Disjunction (the last three counted once each, with Concession split here by resolution status) — are over-represented relative to their natural discourse frequency on purpose: the benchmark's job is to grade the miner where no gold data exists. Both raw and frequency-reweighted metrics must be reported (Section 8) so natural-text performance stays estimable.

The `None` pool deserves its separate line. Our documented failure mode is *over*-connection: a pair-only judgment yields 6–9 relations per atom and invents contradictions on coherent text, which is why mining is response-grounded and there is no ungrounded path. A benchmark that contains only real relations cannot measure that failure at all. The distractors are therefore first-class data, and they must be *in-window* pairs (Section 9, R2) — a pair the candidate policy never scores is not a test of anything.

3.6 Against LoReFact's 78%-false balance

LoReFact makes 4,429 of 5,706 statements logically false by design, and for their task that is defensible: detection is a binary decision, and a detector is trained and measured mostly on positives. Their own Section 2.3 explains the skew as a direct consequence of instructing the generator that "each logical statement must contain a logical error".

For a *graded, graph-level* score the same choice is disqualifying, for three reasons:

(1) **The high end goes unvalidated.** We need to know that a coherent response scores near the top of the range. A corpus that is 78% incoherent contains almost no evidence about that region. The AeroParts family spans only 0.586–0.620, so the interesting variation is *narrow* and sits entirely inside the region such a corpus under-samples.

(2) **A conflict detector would pass.** A degenerate scorer that reports $1 - \mathbb{1}[\text{any contradiction}]$ scores well on a mostly-incoherent corpus and is useless. Only balanced ladders separate "tracks coherence" from "detects conflicts".

(3) **Saturation is invisible.** A readout that saturates low cannot be distinguished from a correct one without coherent items to contrast against.

Adopted composition.

- **Relation validity: 55% valid / 45% invalid.** A *valid* relation is one the text genuinely draws and that genuinely holds; *invalid* covers wrong sense, reversed direction, a factually false endpoint, or a spurious link. This is the *corpus-level* target; P3 asks each family for an explicit 6-of-10 count rather than a percentage, because live models given "55%" over a variable relation count rounded to a convenient fraction instead. $6/10 = 0.60$ sits inside the admission band, so the balance is preserved while the instruction becomes countable.
- **Rung level:** every 5-rung family contains exactly one fully coherent rung and one maximally perturbed rung, so 20% of items sit near the ceiling and 20% near the floor *by construction*. This is what makes ceiling/floor calibration — and hence the dynamic-range check of Section 8.3 — possible at all.

4 The error and perturbation taxonomy

LoReFact has a single error axis: a logical statement is false either because the relation is wrong between correct claims, or because an endpoint claim is false. Our score is graded and graph-level, so we need an axis per mechanism that can move it.

4.1 The sense-confusability matrix

Wrong-sense perturbations should target *confusable* pairs, not random ones. The pairs in Table 10 are seeded from LoReFact's own error analysis (their §4.4) and from our taxonomy's structure. Weight 3 pairs drive item sampling.

4.2 The ten operators

Table 11 defines the operator set and, for each, the expected direction of every readout: $\downarrow$ must fall, $\uparrow$ must rise, $=$ must be invariant within tolerance, $\sim$ unconstrained.

Definitions, briefly: P-WS relabels a gold edge to a confusable sense per Table 10; P-DR swaps source and target on a directed sense; P-EF rewrites a Contrast as an Alternative or the reverse; P-CR deletes the resolver atom, leaving the tension unresolved; P-IC adds one atom contradicting an existing one; P-BC removes a middle link of an entailment chain; P-OS permutes sentences without

Gold	Distractor	Wt	Why confusable, and what it costs
Condition	Precedence/Succession	3	LoReFact's documented failure: "when" read as a time marker. **Coupling-changing** (ENTAILMENT → NONE): the edge disappears
Cause-Effect	Effect-Cause	3	direction only; coupling preserved
Contrast	Alternative	3	exhaustiveness. **Coupling-changing** (CONTRADICTION → EXCLUSIVE)
Alternative	Disjunction	3	which world is excluded: $(1,1)$ vs. $(0,0)$. **Coupling-changing**
Concession	Contrast	3	resolution present or not; coupling preserved, `is_concession` flips
Cause-Effect	Evidence	2	ontic vs. epistemic; coupling preserved
Instantiation	Restatement	2	specificity. **Coupling-changing** (ENTAILMENT → EQUIVALENCE)
Cause-Effect	Condition	2	actual vs. hypothetical
Precedence	Cause-Effect	2	*post hoc* fallacy. **Coupling-changing** (NONE → ENTAILMENT): an edge appears where none belongs
Restatement	Cause-Effect	1	
Disjunction	None	1	

Table 10: Sense confusability. **Coupling-changing** confusions damage the Markov network — a different factor table, or none at all — while coupling-preserving ones damage only interpretability. Section 8.1 requires the two be reported separately, since they have different consequences for a downstream consumer of the score.

changing wording; P-SR asserts an unwarranted link between two atoms the original kept separate; P-DL removes the connective that licensed a gold edge; P-OO rewrites a Precedence as a Succession preserving the actual chronology.

4.3 Two strict invariance tests

These are the sharpest instruments in the design, because the taxonomy's own structure licenses a *falsifiable prediction of no effect* — and a no-effect prediction cannot be satisfied by accident.

Ordering-only invariance (p-oo). `COMPILE[Precedence].level1` = NONE with `ordering_only=True`, so a Precedence↔Succession edit adds **no factor to the network**. All four readouts must be identical up to miner noise; the tolerance is $\varepsilon = 2\sigma$ (Section 5.3). A failure means the miner is leaking temporal cues into truth couplings — which is precisely LoReFact's confirmed failure mode ("when" read as temporal), now measurable as a score-level defect rather than only a label error.

Direction invariance (p-dr). The Markov network is undirected, and `edge_factor_values` is symmetric for a given coupling. So reversing a relation's direction must move *no* readout, while Target-A direction accuracy *must* catch it. This cleanly separates two claims we otherwise conflate: "the miner knows the direction" (an interpretability claim, which we do make) from "the score depends on the direction" (a modeling claim, which we do *not*). Any readout movement under P-DR is a bug, not a finding.

	Operator	Grades	mean_marginal	consistency	log_partition	
1	P-WS wrong sense	A	$\sim$	$\sim$	$\sim$	
2	P-DR direction reversal	A	$=$	$=$	$=$	$\star$
3	P-EF exhaustiveness flip	A, B	$\downarrow$	$\downarrow$	$\downarrow$	
4	P-CR remove resolution	A, B	$\downarrow$	$\uparrow$	$\downarrow$	†
5	P-IC inject contradiction	B	$\downarrow$	$\downarrow$	$\downarrow$	
6	P-BC break causal chain	B	$\downarrow$	$=$	$\downarrow$	
7	P-OS shuffle order	B	$=/\downarrow$	$=$	$=$	
8	P-SR spurious relation	A, B	$\downarrow$	$\downarrow$	$\downarrow$	
9	P-DL drop relation	A	$\uparrow$	$\uparrow$	$\uparrow$	
10	P-OO ordering-only edit	A, B	$=$	$=$	$=$	$\star$

Table 11: Perturbation operators and per-readout expected directions. "Grades" names the target from Section 8: A = per-pair relation labels, B = family ranking. $\star$ marks the two *strict invariance* tests (Section 4.3); † marks the sign-discordant operator (Section 4.4). Rows 9's $\uparrow$ assumes the dropped relation was a conflict; dropping a supporting edge is $\sim$.

4.4 The sign-discordant operator

P-CR (remove resolution) is the single most valuable test in the suite, and it exists only because of Finding 1. Removing a resolution *raises* the effective conflict strength (no 0.45 discount), which lowers `mean_marginal` and `log_partition` while *raising* `consistency`. Encoding that as a per-readout expected direction converts a known theoretical artifact into a regression test: a system that moves all three the same way under P-CR has either lost the discount or lost the distinction between belief-reading and event-reading readouts.

5 Coherence-ordered families

Target B grades the readouts by *ranking*, not by absolute value. The unit is a *family*: a set of responses differing only by controlled perturbations, whose correct coherence order is known by construction.

5.1 Scope: three readouts

LoCoBench grades **three** readouts:

What is excluded, and why it is not an inconsistency. `LCS_METHODS` in `lcs_scorer.py` offers a fourth readout, `reified` — $P(R{=}1)$ for an added coherence node (Eqs. 6–7) — and the scorer will continue to compute it. It is out of *benchmark* scope for three reasons, none of which is a defect in the readout:

(1) **It is not independent of `consistency`.** Both read conflict *activity* rather than belief, so both dip at the concession rung for the same structural reason (Finding 1). Grading both would double-count one signal in every C2 contract and every quirk-conformity figure.

Readout	Reads	Why it is in scope
`mean_marginal`	belief	the shipped default headline (deep-dive Eq. 4): MRF-native, monotone, constant-free, in $[0, 1]$, read straight off Merlin's MAR marginals
`consistency`	event activity	P(no conflict edge jointly active) (Eq. 5). The only in-scope readout that reads conflict *activity* rather than belief, which is what makes the concession quirk of Finding 1 observable at all
`log_partition`	mass	the normalized $\log Z$ readout (Eq. 8), graded in $[0, 1]$ between a contradiction-free ceiling and a saturated floor

Table 12: The three readouts LoCoBench grades. Each reads the same network a different way — belief, event activity, mass — so the set spans the distinction Finding 1 turns on while staying small enough that per-readout ordering contracts remain legible.

(2) **It carries a tunable constant.** The reified node needs a Bernoulli prior (`reified_prior`, default 0.5), so its value is not constant-free the way the three in-scope readouts are. A benchmark number that moves with an unreported hyper-parameter is not comparable across systems.
(3) **It costs inference without adding a distinction.** Dropping it removes the R-node MAR run from the per-item budget while leaving all four *kinds* of Target-B claim — full chain, quirk chain, endpoints, invariance — fully testable.

So the benchmark grades a strict subset of what the code computes. A Phase-2 harness may record `reified` alongside the three as a diagnostic; it simply does not enter any metric, contract or acceptance threshold in this document.

5.2 The five-rung ladder

The rung structure reuses the deep-dive's own variant family (its Table "variants"), whose exact values — brute-forced over 2^{16} worlds — give us a calibrated reference ladder (Table 13).

Rung	Name	Perturbation relative to base	ref. LCS	ref. $\log Z$
0	`worse`	+ a spurious contradiction (P-SR)	0.586	−10.54
1	`base`	as generated; all planned conflicts present	0.607	−9.64
2	`concession_resolved`	resolver present; conflict discounted .80→.55	0.612	−9.64
3	`fix_one_conflict`	drop one unresolved conflict (P-DL)	0.613	−8.95
4	`coherent`	all conflicts removed	0.620	−8.25

Table 13: The reference ladder, with exact `mean_marginal` and $\log Z$ from the AeroParts model. Note how narrow the band is (0.586–0.620, a range of 0.034): the margin rule of Section 5.3 exists because adjacent-rung deltas are of the same order as miner noise.

5.3 Ordering contracts, per readout

Per Finding 1 the three readouts do *not* move together across the rungs, so the ordering contract is stated per readout. Three classes:

C1 — full chain (`mean_marginal`, `log_partition`): $\rho(r_0) < \rho(r_1) < \rho(r_2) < \rho(r_3) < \rho(r_4)$, strict, all four adjacent pairs.
C2 — quirk chain (`consistency`): require $\rho(r_0) < \rho(r_1)$ and $\rho(r_3) < \rho(r_4)$; **assert the inversion** $\rho(r_2) \leq \rho(r_1)$ as a positive prediction; leave $r_2 \to r_3$ unconstrained.
C3 — endpoints (all three readouts): $\rho(r_0) < \rho(r_4)$, strict. The weakest and most robust claim; it survives every quirk, and is therefore the headline ranking number.

The margin rule. Adjacent-rung deltas are small (Table 13), so a constraint is *scored* only when $|\Delta| > 2\sigma$, where σ is the readout's run-to-run standard deviation measured over 5 re-mines of the same rung. Otherwise the constraint is marked `indeterminate` and excluded from the denominator.

This creates an obvious gaming channel that the metrics must close: *a sufficiently noisy system marks everything indeterminate and scores a vacuous 100%.* Section 8.3 therefore requires the indeterminate rate be reported alongside every ranking number, and the two read together.

5.4 Family types

Type	Focus	n	Rung structure
F-CONF	conflicts	40	the reference ladder of Table 13
F-CHAIN	entailment chains	25	P-BC at decreasing depth: break 3 links, 2, 1, 0, then a coherent rewrite
F-EXH	exhaustiveness	20	P-EF walks Alternative → Contrast → resolved; tests whether EXCLUSIVE is scored strictly below CONTRADICTION
F-ORDER	order sensitivity	20	P-OS at decreasing severity (full shuffle, block, adjacent swap, original) *plus* one P-OO rung that must be flat
F-INV	invariance only	15	all five rungs are P-OO/P-DR edits, so the required pattern is a **flat line**

Table 14: Family types. F-INV is the control: any monotone trend there is a false positive, and it guards against a score that merely correlates with edit count rather than with coherence. F-ORDER is dual-purpose, carrying four ordering rungs and one invariance rung in the same family.

5.5 The seed split

The nine fixtures in `data/lcs/` become the hand-authored **dev** split, with the ordering constraints `research_plan.md` §9 already asserts encoded as degenerate two-rung families:
- `example-2-biography` > `example-2-biography-contradicted`;
- `example-5-renda-K` > `example-5-renda-S`;
- `aeroparts-recall` as the reference five-rung family, with the published exact values as `reference_values` (Section 7).

They are dev rather than test for a specific reason given in Section 7.3: the gold edges recoverable from the existing builder are *lossy in sense granularity*, so they can only grade coupling, not sense.

6 The prompt suite

Data generation is five prompts (P1–P5); admission to the benchmark is four more (V1–V4) plus an annotation protocol (V5). This section is the map: Figure 1 shows how they compose, Table 15 gives each prompt's contract, and **the full texts are in Appendix A**, one subsection per prompt, copy-ready.

The suite keeps LoReFact's house style throughout — numbered instructions, few-shot exemplars, bracketed output tags, a `Your Task:` trailer — so the lineage is explicit and the diffs against their Tables 11–14 are legible. The validation prompts keep their function-call idiom from Tables 20–25, which has the useful side effect of making their input/output contract self-declaring. Sense menus match `lcs/prompts.py::_SENSE_MENU` exactly.

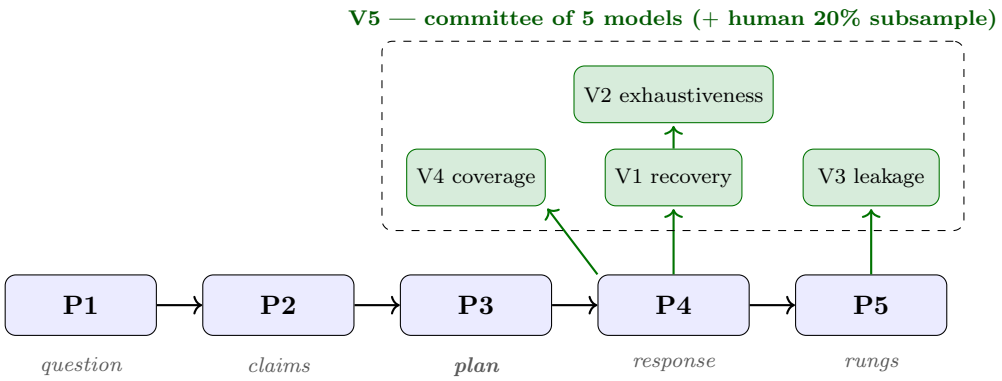

Figure 1: The generation pipeline (blue, left to right) and the admission gates (green, above). P1–P5 produce data; V1–V4 decide whether an item is admitted, and V5 is the committee that runs them. V1 is the load-bearing gate: it re-derives the relation graph from the response *without seeing the plan*, which is what proves the gold is recoverable from the text rather than merely asserted about it. V2 runs on V1's conflict edges only, adjudicating the exhaustiveness boundary that separates CONTRADICTION from EXCLUSIVE.

ID	Role	Inputs	Output	Gate
Generation — these produce the data				
P1	topic → question	`TOPIC`	one bracketed question	—
P2	atomic claims	`QUESTION`	24 tagged claims + 2 holdings	—
p3	**relation plan**	`QUESTION`, `CLAIMS`	**JSON graph**: atoms, relations, non-relations	schema-valid; ≥ 1 each of Alternative, Disjunction, Restatement, Concession, Precedence/Succession
P4	response	`QUESTION`, `PLAN`	prose response, ≥ 500 words	V1, V3, V4
P5	perturbation	`RESPONSE`, `PLAN`, `OPERATOR`	perturbed response + JSON diff	one operator applied; length within 15%
Validation — these decide what is admitted				
V1	relation recovery	`RESPONSE`, `ATOMS`	recovered relation list	**≥80% coupling, ≥70% sense**, else drop the edge or regenerate
V2	exhaustiveness	`CLAIM_A`, `CLAIM_B`, `RESPONSE`	one of A/B/C/D	**unanimity** for an EXCLUSIVE label (Section 9, R1)
V3	naturalness, leakage	`RESPONSE`	3 scores + 3 span lists	all scores ≥ 4; leakage and hedging empty
V4	atom coverage	`RESPONSE`, `ATOMS`	per-atom status + position	100% `asserted`; positions re-verify the window
V5	annotation protocol	—	—	*not a prompt*: the committee and agreement thresholds (Section 6.2)

Table 15: **The prompt suite index.** Full texts in Appendix A. Every `NAME` above is a `[NAME_PLACEHOLDER]` in the prompt body. Note V5 carries no prompt — it is the protocol that runs V1–V4, kept in this series because it is what makes their verdicts binding.

6.1 Why P3 replaces LoReFact's logical-statement stage

The architectural departure is P3. LoReFact generates prose "logical statements" and annotates them afterwards; we generate a *machine-readable relation graph* first and realize it in prose second. So the gold labels are an **input** to generation rather than an output of annotation, which removes a whole class of annotation error — at the cost of requiring V1 to prove the graph is actually recoverable from the text.

That trade is worth making here for a reason specific to our taxonomy. Six of our thirteen senses have no gold data anywhere (Table 3), so post-hoc annotation would be asking annotators to find relation facets they have never seen labelled, at whatever rate those facets happen to occur in generated prose. Planning the graph first makes coverage of the rare facets a *generation constraint* (P3 instruction 5) rather than a matter of luck.

It also changes what LoReFact's 78%-false skew is for. Their skew follows from instructing the generator that every logical statement must contain an error; ours is a `validity` field with an explicit 55/45 target (P3 instruction 8), so the balance is a dial rather than a consequence.

6.2 V5 — the annotation protocol

LoReFact used three graduate-student annotators with a third-annotator tiebreak (reaching 92.18% agreement) plus a five-model committee with unweighted majority voting for logical factuality. Ours mirrors that shape with two additions forced by our design.

Model committee (all items). Five models drawn from at least three distinct families, each running V1, V2 and V4 independently at temperature 0; per-edge label by majority, ties escalated to a human. **Generation/validation disjointness:** the model that ran P3/P4 for an item is *excluded* from that item's committee, per R3.

Human annotation (stratified 20%). Three annotators on 120 items, stratified to cover *all* EXCLUSIVE and CO_NECESSITY gold edges, all resolved Concessions, all P-OO invariance rungs, and a random remainder. These are exactly the facets with no prior art and the highest expected model unreliability, so uniform sampling would waste the human budget.

Statistics and thresholds. Fleiss' κ on the 13-way sense and 6-way coupling labels (human-only and committee-only); Cohen's κ on the binary exhaustiveness decision, reported *separately* as the expected weak point; Krippendorff's α (ordinal) on strength bands; and human-versus-committee agreement, which is what licenses the committee as a human proxy on the other 80%. Acceptance: $\kappa \geq 0.70$ on coupling, ≥ 0.60 on sense, ≥ 0.55 on exhaustiveness. A facet below threshold ships marked `low_agreement` and is excluded from headline metrics but reported as a finding — an unreliable facet is itself a result about the taxonomy.

7 The gold schema

7.1 A benchmark item

The item format is a **strict superset** of `data/lcs/*.json`, so the nine shipped fixtures remain valid and `RelationMiner.mine_from_atoms` loads a benchmark item unchanged. Existing keys (`id`, `name`, `source`, `response`, `num_atoms`, `atoms`, `notes`) keep their meaning; four blocks are added.

```
{
  "id": "locobench-f012-r1",
  "name": "Aviation component failure - base",
  "source": "generated:P4/<model>@<date>",
  "response": "<full response text>",
  "num_atoms": 16,
  "atoms": [
    {"id": "a0", "text": "AeroParts supplied turbine blades to the carrier.",
     "label": "a1", "factual": true, "role": "claim",
     "sentence_index": 1, "char_span": [0, 52]}
  ],
  "notes": "...",

  "relations": [
    {
      "id": "r003",
      "source_id": "a10", "target_id": "a11",
      "level2_sense": "Alternative",
      "level1_coupling": "exclusive",
      "directed": false,
      "exhaustive": true,
      "ordering_only": false,
      "intended_strength_band": "strong",
      "strength_range": [0.85, 1.00],
      "validity": "valid",
      "error_kind": null,
      "is_concession": false,
      "is_resolved_concession": false,
      "resolver_atom_id": null,
      "position_distance": 1,
      "in_candidate_window": true,
      "window_admission": "window",
      "provenance": {
        "planned_by": "P3/<model>",
        "recovered_by": ["<m1>", "<m2>", "<m3>"],
        "recovery_rate": 0.80,
        "exhaustiveness_verdict": "A",
        "exhaustiveness_votes": {"A": 4, "B": 1, "C": 0, "D": 0},
        "human_confirmed": true,
        "annotator_agreement": 1.0,
        "low_agreement": false
      }
    }
  ],

  "non_relations": [
    {"source_id": "a1", "target_id": "a12", "position_distance": 3,
     "in_candidate_window": true,
     "provenance": {"recovered_as_none_by": 5, "human_confirmed": true}}
  ],

  "expected": {
    "family_id": "f012",
    "family_type": "F-CONF",
```

```
    "rung_index": 1,
    "rung_name": "base",
    "perturbation": {"operator": null, "target": null, "parent_rung": null},
    "readout_directions": {
      "mean_marginal": null, "consistency": null, "log_partition": null
    },
    "invariants": [],
    "reference_values": {
      "mean_marginal": 0.607, "consistency": null, "log_partition": -9.64
    }
  },

  "meta": {
    "domain": "Engineering & Technology",
    "topic": "aviation component failure investigation",
    "split": "test",
    "word_count": 623,
    "validation": {
      "v1_coupling_recall": 0.88, "v1_sense_recall": 0.75,
      "v3_fluency": 5, "v3_leakage": [], "v4_coverage": 1.0
    }
  }
}
```


Field notes, in decreasing order of consequence:
`level1_coupling` must satisfy `COMPILE[level2_sense].level1 == level1_coupling`. A builder assertion **must** enforce this, so gold can never silently contradict `taxonomy.py` (Table 8). The same applies to `directed` and `ordering_only`, which mirror `SenseSpec`.
`exhaustive` is meaningful only for CONTRADICTION/EXCLUSIVE edges, where `true` ⇔ Alternative. It is stored *explicitly* rather than derived from the sense: that redundancy is deliberate, because it is what lets a disputed edge be flagged `low_agreement` while still carrying a sense, and it is the field the exhaustiveness metric grades against.
`in_candidate_window` / `window_admission` (`window` | `gate` | `discourse_promoted` | `out_of_window`) are computed at build time by actually running `candidate_pairs.select(policy="gated", window=4)` and recording the verdict. This makes gold recoverability under the shipped pair policy an explicit, queryable property rather than an assumption (R2).
`readout_directions` carries the per-readout expectation from Table 11 for the perturbation that produced this rung — one of `decrease`, `increase`, `invariant`, `unconstrained` per readout, relative to `parent_rung`. It is `null` on a base rung. This is where Finding 1 lives in the data.
`role` on an atom is `claim` | `holding` | `distractor`, so a resolver can be located without re-running the heuristic.

7.2 The family manifest


```
{
  "manifest_version": "1.0",
  "families": [
    {
      "family_id": "f012",
      "family_type": "F-CONF",
      "domain": "Engineering & Technology",
```

```
      "topic": "aviation component failure investigation",
      "rungs": [
        {"index": 0, "name": "worse",  "item_id": "locobench-f012-r0",
         "operator": "spurious_relation", "parent": 1},
        {"index": 1, "name": "base",   "item_id": "locobench-f012-r1",
         "operator": null, "parent": null},
        {"index": 2, "name": "concession_resolved", "item_id": "...-r2",
         "operator": "add_resolution", "parent": 1},
        {"index": 3, "name": "fix_one_conflict",    "item_id": "...-r3",
         "operator": "drop_relation",  "parent": 2},
        {"index": 4, "name": "coherent",            "item_id": "...-r4",
         "operator": "drop_relation",  "parent": 3}
      ],
      "ordering_constraints": [
        {"id": "c1", "class": "C1",
         "readouts": ["mean_marginal", "log_partition"],
         "chain": [0, 1, 2, 3, 4], "strict": true},
        {"id": "c2", "class": "C2",
         "readouts": ["consistency"],
         "required": [[0, 1], [3, 4]],
         "expected_inversions": [[1, 2]],
         "unconstrained": [[2, 3]]},
        {"id": "c3", "class": "C3",
         "readouts": ["mean_marginal", "consistency", "log_partition"],
         "required": [[0, 4]], "strict": true}
      ],
      "invariance_constraints": [],
      "noise_sigma": {"mean_marginal": 0.004, "consistency": 0.011,
                      "log_partition": 0.06},
      "margin_rule": "abs_delta > 2 * noise_sigma else indeterminate"
    }
  ],
  "legacy_pairs": [
    {"family_id": "seed-renda",
     "rungs": [{"index": 0, "item_id": "example-5-renda-S"},
               {"index": 1, "item_id": "example-5-renda-K"}],
     "ordering_constraints": [{"id": "c3", "class": "C3",
       "readouts": ["mean_marginal"], "required": [[0, 1]], "strict": true}]},
    {"family_id": "seed-biography",
     "rungs": [{"index": 0, "item_id": "example-2-biography-contradicted"},
               {"index": 1, "item_id": "example-2-biography"}],
     "ordering_constraints": [{"id": "c3", "class": "C3",
       "readouts": ["mean_marginal"], "required": [[0, 1]], "strict": true}]}
  ]
}
```


`expected_inversions` in the `c2` constraint is Finding 1 made executable: the harness *asserts* that `consistency` falls from rung 1 to rung 2, rather than exempting the step.

7.3 Migrating the seed fixtures

`scripts/build_lcs_examples.py` already contains hand-authored gold relations, as 4-tuples `(src_label, trg_label, level1_type, relation_class)` with `relation_class` $\in$

{prerequisite, invalidation} — and _build_record **discards them**. The comment at its definition promises they are "expanded to a0-based ids at build time"; that expansion was never implemented, so the data is live only inside the script. Phase 1 specifies surfacing it:

- prerequisite → level2_sense: "Cause-Effect", level1_coupling: "entailment", directed: true;
- invalidation → "Contrast", CONTRADICTION, directed: false;
- the AeroParts equivalence edge → "Restatement", EQUIVALENCE;
- the AeroParts blame conflict additionally gets is_concession: true, is_resolved_concession: true and its resolver_atom_id, per the fixture's existing notes.

Why this makes the seed set dev, not test. The migration is **lossy in sense granularity**: prerequisite covers whatever the true sense was, so mapping it to Cause-Effect is a guess wherever the truth is Evidence, Condition or Instantiation. Migrated edges are therefore graded on **coupling only** unless a human re-annotates the sense. Migrated edges carry provenance.planned_by: "hand-authored" and must still pass V1 to be scored.

Two housekeeping items to resolve before Phase 2 touches that script. data/lcs/example-6-incident.json is **orphaned**: the builder registers only eight examples and does not know it, and data/lcs/README.md omits it. Separately, that README has been **hand-edited since generation** — it documents the two-stage node_priors path that _write_readme does not emit — so re-running the builder would regress it. Either fold example 6 into the builder and re-sync the README generator, or move both under the new benchmark builder.

8 Metrics

Notation: item k has gold edge set G_k and candidate-pair set C_k; a system produces predicted edges P_k. Let G^{edge} be the gold edges with a non-NONE coupling, and N the non_relations pool. Write c, s, d for gold coupling/sense/direction and $\hat{c}, \hat{s}, \hat{d}$ for predictions.

All Target-A metrics are computed over candidate pairs only. A pair the policy never scored cannot be a miner error. Gold edges with in_candidate_window: false are excluded from recall and reported separately as the **structural-miss rate** — conflating the two would blame the miner for the gate's decisions (R2).

8.1 Target A — the miner as a classifier

A1. Coupling accuracy. CouplingAcc $= |\{e \in G^{\text{edge}} : \hat{c}(e) = c(e)\}| \,/\, |G^{\text{edge}}|$.

A2. Sense accuracy. Same denominator with $\hat{s}, s$. Report also **SenseAcc-hard**, restricted to edges whose confusability weight is 3 (Table 10), and split every figure into *coupling-changing* versus *coupling-preserving* confusions.

A3. Direction accuracy. Over gold edges where COMPILE[s].directed holds *and* the coupling was predicted correctly (direction on a wrong coupling is not meaningful):

$$\text{DirAcc} = \frac{|\{e : \hat{d}(e) = d(e) \wedge \hat{c}(e) = c(e)\}|}{|\{e \in G : \texttt{directed}(s(e))\}|}.$$

A4. None-detection P/R/F1. With P_{none} the pairs the system labels NONE:

$$\mathcal{P} = \frac{|N \cap P_{\text{none}}|}{|P_{\text{none}}|}, \qquad \mathcal{R} = \frac{|N \cap P_{\text{none}}|}{|N|}, \qquad F_1 = \frac{2\mathcal{P}\mathcal{R}}{\mathcal{P}+\mathcal{R}}.$$

Note $1 - \mathcal{P}$ **is** the over-connection rate — our documented failure mode, and the reason the `None` pool is first-class data (Section 3.5).

A5. Exhaustiveness accuracy. Over gold edges with coupling $\in$ {CONTRADICTION, EXCLUSIVE}, the rate at which `exhaustive` is predicted correctly, *plus* the explicit 2×2 Contrast/Alternative confusion matrix. This metric has no LoReFact analogue and is the one that justifies EXCLUSIVE as a separate coupling.

A6. Concession-resolution accuracy. Over gold `is_concession` edges: the binary "did the system mark it resolved", reported separately from exact `resolver_atom_id` match.

A7. Strength-band agreement. Exact-match rate; *adjacent* accuracy (off-by-one band counted correct); Krippendorff's α (ordinal); and the mean absolute deviation of the predicted p from the nearest edge of the gold band, zero inside the band.

A8. Confusion matrices. 13×13 over senses and 6×6 over couplings, each with an extra `structural` row/column for out-of-window gold. Published as the primary diagnostic artifact — the aggregate numbers above are summaries of it.

8.2 LoReFact-comparable analogues

LoReFact's three headline logic metrics, lifted to a graded, graph-level setting (Table 16). The point of this table is comparability: a reader of both papers should be able to line the numbers up.

LoReFact	LoCoBench	Definition and difference
LogicRecall	**EdgeRecall**	$\lvert\{e \in G^{\text{edge}} : \hat{c}(e) \neq \text{NONE}\}\rvert / \lvert G^{\text{edge}}\rvert$ — fraction of in-window gold edges detected as *some* relation. Complements $\mathcal{R}$ above
LogicTypeAcc	**CouplingAcc**, **SenseAcc**	their 5-class becomes our 6-class coupling and 13-class sense, i.e. two granularities instead of one
LogicFactualityAcc	**ValidityAUROC**, **BandMAE**	their binary "does the dependency hold" becomes graded: use the miner's edge probability p as a score for `validity` and report AUROC. A thresholded **ValidityAcc** at p=0.5 is also reported for direct numeric comparison
—	**CoherenceAUC**	new: pooling all items, each readout used as a score for "this rung is in the top half of its family". One number per readout, comparable across readouts *and* against DiscoScore / G-Eval / SelfCheckGPT-NLI

Table 16: LoReFact metric analogues. The graded-versus-binary shift in the third row is the substantive one: a binary logic-factuality label discards exactly the information our relation probability carries.

8.3 Target B — readout ranking

For readout ρ and family f with rungs $r_0..r_4$, and σ_ρ the measured run-to-run noise:

B1. Pairwise ranking accuracy, over the constraint's required pairs, excluding indeterminate ones:

$$\mathrm{PRA}(\rho) = \frac{\sum_f |\{(i,j) \text{ required} : \rho(r_j) - \rho(r_i) > 2\sigma_\rho\}|}{\sum_f |\{(i,j) \text{ required} : \text{determinate}\}|}.$$

Must be reported with the indeterminate rate. A system can score PRA = 1 trivially by being noisy enough that every pair is indeterminate; the two numbers are only meaningful together.

B2. Kendall τ-b between the readout's rung ordering and the gold rung index, per family, averaged. Computed on the full C1 chain for `mean_marginal` and `log_partition`; on the C2-required sub-chain only for `consistency` since τ over the full chain is not meaningful given the predicted inversion.

B3. Monotonicity violation rate. Fraction of C1 adjacent pairs moving the wrong way by more than $2\sigma_\rho$. Strict violations only — noise-level wobble is not a violation.

B4. Quirk-conformity rate (C2, `consistency` only). Fraction of families where the readout *does* invert at rung $1 \to 2$ as Finding 1 predicts. **Target is high, not low**: a system monotone here is not implementing the concession discount.

B5. Invariance pass rate. Over P-OO and P-DR rungs, $\mathrm{INV}(\rho) = |\{\,|\rho(\text{perturbed}) - \rho(\text{parent})| \le 2\sigma_\rho\}|/|\text{items}|$, reported separately as INV-ORDERING and INV-DIRECTION. This is the one metric where all three readouts share the same strict expectation; a INV-DIRECTION failure means the factor tables have become direction-sensitive.

B6. Endpoint separation and dynamic range. $\rho(r_4) - \rho(r_0)$, mean and distribution, plus the 5th/95th percentile of ρ over all items. The reference family spans only 0.586–0.620, so a *compressed* range is expected and must be quantified rather than treated as a defect.

B7. Human correlation. Spearman ρ between each readout and human coherence ratings on the 120-item human subsample, against DiscoScore [DiscoScore], the G-Eval coherence dimension [G-Eval] and SelfCheckGPT-NLI [SelfCheckGPT], per `research_plan.md` §9.

Metric	`mean_marginal`	`consistency`	`log_partition`
PRA, C1 chain	✓	—	✓
PRA, C2 required	✓	✓	✓
Kendall τ	full	sub-chain	full
Monotonicity violation	✓	—	✓
Quirk conformity	—	✓	—
Invariance (P-OO, P-DR)	✓	✓	✓
Endpoint separation (C3)	✓	✓	✓
Human Spearman	✓	✓	✓

Table 17: Metric applicability by readout, following from the C1/C2/C3 contract classes of Section 5.3. Target-A metrics are deliberately absent: they grade the miner upstream of scoring, so a miner regression is diagnosable without re-running inference.

9 Risks and Phase-2 open questions

R1 — LLM-generated exhaustiveness may be unreliable (highest risk). Deciding Contrast versus Alternative requires quantifying over possible worlds, which models handle poorly, and it is exactly the boundary where EXCLUSIVE earns its existence as a separate coupling. Mitigations: V2's explicit "coherent third state" procedure; **unanimity** (not majority) of the committee required for an EXCLUSIVE gold label; 100% human coverage of those edges in the subsample; `low_agreement` flagging. *Fallback:* seed Alternative edges from **structurally guaranteed** pairs — numeric or polarity complements ("no one was harmed" / "three people died"), where exhaustiveness follows from the semantics of negation rather than from world knowledge. *Open:* if human κ on exhaustiveness lands below 0.55, is EXCLUSIVE a defensible benchmark facet at all, or should it ship as a research-only subset?

R2 — the candidate window versus gold recoverability. Under `policy="gated", window=4`, a gold edge whose endpoints are more than 4 positions apart and which fails both the similarity gate and the discourse-adjacency check is **unmineable by construction**. Handled in three places: P3 instruction 4 biases generation into the admissible set; build time records `in_candidate_window` / `window_admission` per edge by running the real selector; the metrics exclude out-of-window gold and report the structural-miss rate separately. *Open:* P3 constrains *planned* positions but P4's realized order may drift, so V4's `position` output must re-verify the constraint post hoc and items that break it must regenerate. *Also open:* should Phase 2 ship a deliberate `long_range` subset of out-of-window gold to grade *the gate itself* as a retrieval problem (precision/recall of admission)? That is a different and probably valuable evaluation, and the schema already supports it.

R3 — self-generation bias. If the same model family generates (P3/P4) and mines (Prompt A/B), the miner may recover its own lexical fingerprints rather than genuine discourse structure, inflating every Target-A number. Mitigations: per-item generator exclusion from the validation committee (Section 6.2); the generator recorded in `source` and stratified across ≥ 3 families so per-generator breakdowns are computable; and a **cross-generation control** — Target-A reported both pooled and restricted to items whose generator family differs from the miner's, with the gap published as a bias estimate. *Open (recommended yes):* a small human-authored slice, say 50 items with no LLM generation at any stage, as an unbiased anchor. It is cheap relative to its diagnostic value, and the nine existing fixtures are a precedent for the format.

R4 — Restatement's strength prior. Per Finding 2 (Section 2.2) the 0.90 prior is unreachable on the production path, so it cannot be tested through the pipeline. What Phase 2 *can* report is the distribution of mined strength on gold Restatement edges — i.e. whether 0.90 is empirically justified. *Open:* if mined Restatement strength centres well away from 0.90, the prior should be re-estimated or removed rather than left as unreachable code.

R5 — the MLN formulation cannot be benchmarked end to end. `formulation="mln"` constructs but raises at `score()`, and EXCLUSIVE / CO_NECESSITY raise in that path specifically. Target B is therefore scoped to the MRF. The MLN's closed-form pairwise fragment is verified to reproduce the MRF exactly, so nothing is lost for the pairwise facets; beyond-pairwise structure is out of scope until an engine exists.

R6 — generation cost. LoReFact reports ≈110 LLM calls and ≈$0.32 per sample for their *evaluation* pipeline alone. Ours adds a 5-model validation committee over V1/V2/V4 and a 5-rung ladder per family. Phase 2 should cost the run before committing to 600 items, and the schema's `reference_values` exist partly so that cheap dry-run scoring (the brute-force oracle, and `score_all`'s shared base inference) can substitute during harness development.

On the inference side the three-readout scope (Section 5.1) costs **5 Merlin calls per item**, not 6: a base MAR and a base PR shared by all three, plus `consistency`'s U-chain MAR and `log_partition`'s contradiction-free ceiling PR and saturated-floor MAP. Excluding `reified` removes exactly its R-node MAR run. Over 600 items × 5 rungs that is 3,000 items and ≈15,000 Merlin invocations, so the saving is real but second-order next to the LLM committee cost above.

10 Summary of Phase-1 decisions

1. **Two gradable targets**, not one: per-pair relation labels (A) grade the miner as a classifier; coherence-ordered families (B) grade the three readouts by ranking. A alone would leave the score untested; B alone would give no diagnosis of *which* facets fail.
2. **Thirteen senses, five couplings**, exactly as `taxonomy.py` defines them, with EQUIVALENCE, EXCLUSIVE and CO_NECESSITY over-represented because no prior gold data exists for them.
3. **Two orthogonal organizations, named apart.** *Facets* are LoCoBench's own structural axes (relation, item, annotation); *topic* and *domain* stay reserved for the 36 subjects inherited from LoReFact, **all of which must be represented**. Both are coverage requirements, and both are reported — a topic breakdown says whether the benchmark generalizes across subject matter, a facet breakdown says which structural phenomena a scorer handles.
4. **Validity-balanced, not false-skewed**: 55/45 valid/invalid with a coherent and a maximally-perturbed rung in every family, departing deliberately from LoReFact's 78% because a graded score must be validated at both ends.
5. **A relation plan replaces prose logical statements**: gold is an *input* to generation, gated by a round-trip recovery check rather than produced by post-hoc annotation.
6. **Per-readout ordering contracts** (C1/C2/C3), because one of the three readouts provably invert at the concession rung and a global monotonicity requirement would fail a correct implementation.
7. **Two strict invariance tests** — ordering-only and direction-reversal — which are the design's sharpest instruments because a prediction of *no effect* cannot be satisfied by accident.
8. **A superset schema**, so the nine shipped fixtures become the dev split with no format fork, graded on coupling only where their migration is lossy.

Phase 2 is the generation run: topic selection, the live prompt pipeline, the validation committee, and the harness that turns `expected` blocks into assertions.

References

[LoReFact] Q. Xie, W. Zheng, X. Shen, and R. Xia. *LoReFact: Bridging the Logic Gap in Fact-Checking.* Findings of the Association for Computational Linguistics: ACL 2026, pages 6974–6996, 2026.

[FactReasoner] R. Marinescu et al. *FactReasoner: A Probabilistic Approach to Long-Form Factuality Assessment for Large Language Models.* arXiv:2502.18573, 2025.

[DiscoScore] W. Zhao, M. Strube, and S. Eger. *DiscoScore: Evaluating Text Generation with BERT and Discourse Coherence.* arXiv:2201.11176, 2023.

[G-Eval] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu. *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment.* arXiv:2303.16634, 2023.

[SelfCheckGPT] P. Manakul, A. Liusie, and M. J. F. Gales. *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.* arXiv:2303.08896, 2023.

[PDTB3] R. Prasad, B. Webber, and A. Lee. *Penn Discourse Treebank Version 3.0.* LDC2019T05, Linguistic Data Consortium, 2019.

[Barzilay] R. Barzilay and M. Lapata. *Modeling Local Coherence: An Entity-Based Approach.* Computational Linguistics, 34(1):1–34, 2008.

[Li] J. Li and D. Jurafsky. *Neural Net Models of Open-Domain Discourse Coherence.* EMNLP 2017.

[SAFE] J. Wei et al. *Long-Form Factuality in Large Language Models.* NeurIPS 2024.

A The prompts, verbatim

The nine prompts of Table 15, copy-ready. Each is preceded by its contract: the placeholders it takes, the output it must produce, and the gate its output must pass. Generation prompts (P1–P5) come first, then validation (V1–V4). V5 is not a prompt and is specified in Section 6.2.

A.1 P1 — topic to question

P1 — topic to question
Inputs: `[TOPIC_PLACEHOLDER]` — one of the 36 subject topics of Table 6. The prompt is run per topic, so *topic selection is not the prompt's job*: instruction 4's preference for adjudicated framings applies *within* whatever topic it is given, and never substitutes for covering all 36 (Section 3.2).
Output: One question, wrapped in square brackets, no preamble.
Gate: None. A weak question is caught downstream by P3 failing instruction 5.

```
Instructions:
1. You are given a TOPIC. Generate one open-ended QUESTION about that topic.
2. The QUESTION must begin with Why, How, What, Does, or Which.
3. The QUESTION must elicit an answer containing MULTIPLE, DIVERSE logical
   relationships. Aim for an answer that would naturally contain at least four
   of the following: a causal chain; a condition; a temporal ordering; an
   evidential appeal; a general claim with a specific instance; a restatement
   or definitional equivalence; a non-exhaustive contrast; an EXHAUSTIVE pair
   of competing alternatives of which EXACTLY ONE can hold; a disjunctive pair
   of which AT LEAST ONE must hold; a tension that an authoritative finding
   later resolves.
4. Prefer topics where competing explanations are adjudicated: incident
   investigations, litigation, diagnostic reasoning, attribution disputes,
   competing scientific hypotheses. Such topics naturally produce exhaustive
   alternatives and resolving holdings.
5. The QUESTION must be answerable in 500-900 words of formal prose.
6. The QUESTION must not itself contain the words "cause", "condition",
   "alternative", or "contrast", so the answer's relations are not primed
   lexically.
7. Do NOT ask a multi-part question joined by "and" - one question only.
8. Wrap the QUESTION in square brackets.
```

```
9. Output the bracketed QUESTION only, with no preamble.

Use the following examples to better understand your task.

TOPIC: aviation component failure investigation
QUESTION: [How do investigators determine responsibility when a component
supplied by a third party fails in service and the operator and the
manufacturer offer competing explanations?]

TOPIC: antibiotic resistance in hospital settings
QUESTION: [Why does resistance emerge in some hospital units but not others,
and how is a specific outbreak traced to its origin?]

Your Task:
TOPIC: [TOPIC_PLACEHOLDER]
QUESTION:
```

Instructions 3 and 4 are the substantive extension of LoReFact's Table 11, which asks only for "causal, conditional, or temporal reasoning". Instruction 4 encodes an empirical bet: adjudicated domains are where exhaustive alternatives and resolving holdings occur naturally, so steering topic selection is cheaper than forcing the structure later. Instruction 6 prevents lexical priming that would make the miner's job artificially easy.

A.2 P2 — atomic claim generation

P2 — atomic claim generation

Inputs: `[QUESTION_PLACEHOLDER]`.
Output: A markdown code block: 24 claims as `-` bullets, each with one of the 9 tags of Table 5, plus 2 `[holding]` claims.
Gate: Composition must match instruction 3 exactly (14 correct, 4 incorrect, and the alt/disj/equiv pairs); the holdings must use adjudicating vocabulary.

```
Instructions:
1. You are given a QUESTION. Generate a set of ATOMIC CLAIMS that a
   well-informed answer to it might contain.
2. An atomic claim is a sentence containing a singular piece of information,
   which should be basic, indivisible, and independently verifiable.
3. Generate exactly 24 atomic claims, in the following required composition:
   (a) 14 factually correct claims;
   (b) 4 factually incorrect claims (plausible but false);
   (c) 2 EXHAUSTIVE ALTERNATIVE claims: a pair X, Y such that exactly one of X
       and Y must be true - they cannot both hold, and they cannot both fail.
       Example pair: "No one was harmed in the incident." / "Three people died
       in the incident."
   (d) 2 DISJUNCTIVE claims: a pair X, Y such that at least one must be true,
       but both may be true simultaneously. Make each one a STATE OF THE WORLD,
       not a finding or a check that reported something: "the alloy was
       substituted" and "the batch was mislabelled", not "assay A flagged it"
       and "assay B flagged it".
   (e) 2 EQUIVALENT claims: a pair X, Y asserting the same fact in different
       words, so that either one being true makes the other true.
4. Additionally generate 2 HOLDING claims: sentences in which an authority
   (a court, tribunal, regulator, final report, or review board) reaches a
```

```
   determination that settles a dispute between two other claims in your list.
   A holding claim must use adjudicating language - "held", "ruled",
   "concluded", "determined", "found that", "ultimately".
5. Each claim must be a single declarative sentence with no connective linking
   it to another claim. Do NOT use "because", "although", "therefore", "if",
   "while", "however". Relations are added at a later stage, not here.
6. Do NOT indicate which claims are factually incorrect in their wording; do
   NOT hedge with "assume", "might", "possibly", or "allegedly".
7. Output as a markdown code block: an unordered list with "-" bullets. Append
   a tag in square brackets to every claim, one of: [correct], [incorrect],
   [alt-pair-1], [alt-pair-2], [disj-pair-1], [disj-pair-2], [equiv-pair-1],
   [equiv-pair-2], [holding].

Use the following example to better understand your task.

QUESTION: [How do investigators determine responsibility when a component
supplied by a third party fails in service and the operator and the
manufacturer offer competing explanations?]

ATOMIC CLAIMS:
‘‘‘
- AeroParts supplied turbine blades to the carrier. [correct]
- The turbine blades were installed in the carrier’s A-320 fleet. [correct]
- Seventeen turbine blades failed in flight during service. [correct]
- The regulator opened a safety probe into the blade failures. [correct]
- Maintenance logs recorded abnormal vibration on the failed blades. [correct]
- The airline argued that the failures were caused by pilot error. [correct]
- The final report concluded the root cause was a metallurgical defect. [correct]
- The regulator issued an order grounding the fleet. [correct]
- Every affected aircraft was pulled from service. [equiv-pair-1]
- The regulator’s order removed all affected aircraft from operation. [equiv-pair-2]
- The blades were certified under a 1998 airworthiness standard. [incorrect]
- No one was harmed in the incidents. [alt-pair-1]
- Three passengers died in one of the failures. [alt-pair-2]
- The alloy supplied for the blades was substituted. [disj-pair-1]
- The blade batch was mislabelled at the mill. [disj-pair-2]
- The tribunal held that AeroParts was liable for the blade failures. [holding]
- The accident report found the pilots not at fault. [holding]
‘‘‘

Your Task:
QUESTION: [QUESTION_PLACEHOLDER]
ATOMIC CLAIMS:
```

Instruction 3(c)–(e) and instruction 4 are the extension. LoReFact’s Table 12 asks for 10 correct and 10 incorrect claims and nothing else; its instruction that claims be “independently verifiable” actively *precludes* the equivalent pairs an EQUIVALENCE edge needs. The tagged pairs are the raw material for the three couplings with no prior gold, and the `[holding]` claims are what make a *resolved* Concession constructible. Their required vocabulary is not stylistic: it matches the cue list in the miner’s `_looks_like_holding` heuristic (`held`, `ruled`, `concluded`, `tribunal`, `determined`, . . .), so a planned resolution is one the shipped resolver can actually find. Instruction 5 is load-bearing in the other direction: claims must arrive *connective-free* so that every relation in the final response traces to P3’s plan rather than to an accident of claim phrasing.

A.3 P3 — relation plan generation

P3 — relation plan generation (the architectural departure, Section 6.1)
Inputs: `[QUESTION_PLACEHOLDER]`, `[CLAIMS_PLACEHOLDER]` (P2s tagged list).
Output: One JSON object: `atoms` (with `pos`), `relations` (sense, strength band, validity, error kind, resolver), `non_relations`.
Gate: Schema-valid; $\geq$ 1 each of Alternative, Disjunction, Restatement, Concession and Precedence/Succession (instr. 5); 6 valid / 4 invalid of 10 relations (instr. 8). Locality (instr. 4) is *recorded, not enforced* here — see R2.

This replaces LoReFact's logical-statement stage (their Table 13). Their stage emits prose and requires that "each logical statement must contain a logical error", which is what produces their 78% skew. Ours emits a graph and controls the valid/invalid ratio explicitly.

```
Instructions:
1. You are given a QUESTION and a tagged list of ATOMIC CLAIMS. Produce a
   RELATION PLAN: a machine-readable graph of the logical relations a response
   should draw between those claims.
2. Select 14-18 claims to use. Aim for 16. Number them with FRESH consecutive
   positions: the first claim you assert is pos 1, the next pos 2, and so on
   with NO gaps and NO repeats, ending at pos N where N is how many you
   selected. Do NOT carry over the numbering of the input list - if you pick the
   3rd, 7th and 12th claims they become pos 1, 2 and 3. The position is the
   order the response will assert the claim in. Every selected claim keeps its
   text EXACTLY as given. Do NOT introduce information not in the claims, and
   do NOT alter the claims.
3. Plan EXACTLY 10 RELATIONS. Each relation connects exactly two selected claims
   and has a SENSE, one of: Cause-Effect, Effect-Cause, Evidence, Condition,
   Restatement, Instantiation, Contrast, Concession, Alternative, Disjunction,
   Precedence, Succession.
   - Cause-Effect: source causes target. Effect-Cause: source is the effect,
     target its cause.
   - Evidence: source is evidence for target. Condition: source is a condition
     for target.
   - Restatement: source and target assert the same thing.
   - Instantiation: source is a general claim, target a specific instance.
   - Contrast: opposition that is NOT exhaustive - both could be false.
   - Concession: a tension the text itself resolves.
   - Alternative: EXHAUSTIVE competing options - EXACTLY ONE holds. Not both,
     and not neither. Use your [alt-pair-*] claims here.
   - Disjunction: AT LEAST ONE holds; both may hold. Use your [disj-pair-*]
     claims here.
   - Precedence / Succession: ordered in time with NO truth dependence -
     neither claim being true or false says anything about the other.
4. Keep relations LOCAL: prefer pairs within 4 POSITIONS of each other, so the
   graph stays recoverable from the response by a local reader. A longer-range
   pair is acceptable when the two claim texts share a named entity, and a few
   long-range links (for instance a closing claim answering the opening one) are
   fine. Order the positions so that related claims sit near each other.
5. The plan must include AT LEAST ONE relation of each of these senses:
   Alternative, Disjunction, Restatement, Concession, and one of
   Precedence/Succession.
6. Every Concession must be marked resolved or unresolved. A resolved
   Concession must name a RESOLVER: a [holding] claim, also selected and
```

positioned AFTER both endpoints.
7. Give each relation an intended STRENGTH BAND: strong (0.85-1.00), moderate (0.60-0.84), or weak (0.35-0.59) - how strongly the source determines the target under that sense. Restatement relations are always strong.
8. Give each relation an intended VALIDITY: valid if the relation genuinely holds between the claims as stated; invalid if it does not. Of your 10 relations, mark EXACTLY 6 "valid" and EXACTLY 4 "invalid" - count them before you emit, and do not compute a percentage. For each invalid relation give an ERROR_KIND, one of: wrong_sense, wrong_direction, false_endpoint, spurious.
9. Also list 4-6 NON-RELATIONS (aim for 5): pairs of selected claims within 4 positions of each other between which the response must draw NO logical dependence. These are the distractors a relation miner must correctly label None. A non-relation pair must NOT also appear in your relations list, in either direction - the two lists are disjoint.
10. Output a single JSON object in a code block and nothing else. Below is a COMPLETE, correctly-shaped example - copy its structure exactly, not its text. It has 14 atoms numbered 1..14 with no gaps, 10 relations (6 valid, 4 invalid), all five required senses, and 5 non-relations disjoint from the relations:

```json
{
  "atoms": [
    {"pos": 1, "text": "<claim text>"}, {"pos": 2, "text": "<claim text>"},
    {"pos": 3, "text": "<claim text>"}, {"pos": 4, "text": "<claim text>"},
    {"pos": 5, "text": "<claim text>"}, {"pos": 6, "text": "<claim text>"},
    {"pos": 7, "text": "<claim text>"}, {"pos": 8, "text": "<claim text>"},
    {"pos": 9, "text": "<claim text>"}, {"pos": 10, "text": "<claim text>"},
    {"pos": 11, "text": "<claim text>"}, {"pos": 12, "text": "<claim text>"},
    {"pos": 13, "text": "<claim text>"}, {"pos": 14, "text": "<claim text>"}
  ],
  "relations": [
    {"source_pos": 1, "target_pos": 2, "sense": "Cause-Effect",
     "strength_band": "strong", "validity": "valid",
     "error_kind": null, "resolved": null, "resolver_pos": null},
    {"source_pos": 2, "target_pos": 3, "sense": "Evidence",
     "strength_band": "moderate", "validity": "valid",
     "error_kind": null, "resolved": null, "resolver_pos": null},
    {"source_pos": 3, "target_pos": 4, "sense": "Restatement",
     "strength_band": "strong", "validity": "valid",
     "error_kind": null, "resolved": null, "resolver_pos": null},
    {"source_pos": 5, "target_pos": 6, "sense": "Alternative",
     "strength_band": "strong", "validity": "valid",
     "error_kind": null, "resolved": null, "resolver_pos": null},
    {"source_pos": 7, "target_pos": 8, "sense": "Disjunction",
     "strength_band": "moderate", "validity": "valid",
     "error_kind": null, "resolved": null, "resolver_pos": null},
    {"source_pos": 9, "target_pos": 10, "sense": "Concession",
     "strength_band": "moderate", "validity": "valid",
     "error_kind": null, "resolved": true, "resolver_pos": 11},
    {"source_pos": 11, "target_pos": 12, "sense": "Precedence",
     "strength_band": "weak", "validity": "invalid",
     "error_kind": "wrong_sense", "resolved": null, "resolver_pos": null},
    {"source_pos": 12, "target_pos": 13, "sense": "Cause-Effect",
```

```
     "strength_band": "moderate", "validity": "invalid",
     "error_kind": "wrong_direction", "resolved": null, "resolver_pos": null},
    {"source_pos": 13, "target_pos": 14, "sense": "Contrast",
     "strength_band": "weak", "validity": "invalid",
     "error_kind": "spurious", "resolved": null, "resolver_pos": null},
    {"source_pos": 4, "target_pos": 6, "sense": "Instantiation",
     "strength_band": "weak", "validity": "invalid",
     "error_kind": "false_endpoint", "resolved": null, "resolver_pos": null}
  ],
  "non_relations": [
    {"source_pos": 1, "target_pos": 4}, {"source_pos": 2, "target_pos": 5},
    {"source_pos": 6, "target_pos": 9}, {"source_pos": 8, "target_pos": 11},
    {"source_pos": 10, "target_pos": 13}
  ]
}
```

```
Your Task:
QUESTION: [QUESTION_PLACEHOLDER]
ATOMIC CLAIMS: [CLAIMS_PLACEHOLDER]
RELATION PLAN:
```

Instruction 4 *biases* generation toward recoverability and exists because of R2 (Section 9): the default candidate policy is `gated` with `window=4`, so a planned edge whose endpoints are farther apart and which fails both the similarity gate and the discourse-adjacency check is *unmineable by construction* and cannot be a fair miner error. It is deliberately **not** an admission gate — R2 assigns the authoritative check to build time, where the real candidate selector runs on the *realized* text and records `window_admission` per edge, and the metrics then exclude out-of-window gold and report the structural-miss rate separately. Enforcing it twice was measurably harmful: on the first live runs it was the largest single cause of plan-stage rejection, and the edges it caught were mostly a closing claim referring back to the opening one, which is ordinary discourse structure rather than an error. Instruction 5 guarantees per-item coverage of the no-prior-gold facets. Instruction 9 is what makes the over-connection rate measurable at all.

Instructions 2, 8 and 9 are worded against the failure modes observed on live models rather than for brevity. Positions are described as *fresh consecutive* integers because the parser requires $1..N$ with no gaps and a model that reuses the input list's numbering fails immediately; instruction 8 states an explicit count (6 valid, 4 invalid of 10) because "55% valid" over a variable 8–12 relations invited models to round to a convenient fraction — one returned exactly 3/4 on four families out of five; and instruction 9 states the relation/non-relation disjointness that the parser has always enforced silently. The corpus-level 55/45 balance of Section 3.6 is unchanged in intent: 6-of-10 is 0.60, comfortably inside the admissible band.

A.4 P4 — response generation

P4 — response generation
Inputs: `[QUESTION_PLACEHOLDER]`, `[PLAN_PLACEHOLDER]` (P3s JSON).
Output: The response prose in a code block, ≥ 500 words.
Gate: v1 (relations recoverable), v3 (no leakage, no hedging), v4 (100% of atoms asserted).

```
Instructions:
1. You are given a QUESTION and a RELATION PLAN in JSON. Write a RESPONSE that
```

answers the question and realizes the plan exactly.
2. Assert every planned atom EXACTLY ONCE, in the plan's POSITION order, preserving each atom's factual content. Do not restate an atom you have already asserted in order to attach a later relation to it - refer back with a pronoun or a short noun phrase instead. Name that phrase after the thing the atom is about, not after its status as a finding: "the primitive carpal morphology" or "that wrist evidence", not "the analysis's flagging of primitive traits"; "these two accounts" or "either possibility", not "the two claims in circulation". A summary phrase covering both endpoints of one connective is fine - "at least one of these must hold: X, or Y". You may adjust surface wording for fluency but must not change what is asserted. Never let one atom absorb another: each must stay separately true or false. Joining two atoms with a connective is fine and often required - "either X or Y" and "although X, Y" both assert two atoms and are exactly what instruction 3 asks for.
3. Realize every planned RELATION so a careful reader could recover it from the text alone, using language appropriate to its sense:
   - Cause-Effect / Effect-Cause: "caused", "led to", "resulted from", "because"
   - Evidence: "indicates", "confirms", "as shown by"
   - Condition: "if", "provided that", "only when"
   - Restatement: "that is", "in other words", "equivalently"
   - Instantiation: "for example", "specifically", "in one case"
   - Contrast: "by contrast", "whereas", "on the other hand"
   - Concession: "although", "despite", "even though"
   - Alternative: make the exhaustiveness EXPLICIT - "either ... or", "one of these accounts must be wrong", "the two cannot both be true, and one of them must hold"
   - Disjunction: make the at-least-one reading EXPLICIT - "at least one of", "one or both", "perhaps both". Say "perhaps both", never "possibly both".
   - Precedence / Succession: "before", "after", "subsequently" - and make clear the ordering carries NO causal or inferential force. Do NOT write "and therefore" or "which led to" around these.
4. For each planned NON-RELATION, assert both atoms but draw no dependence between them. Separate them by DISTANCE and PARAGRAPHING, not by a marker: put other material between them, or place them in different paragraphs, and simply use no connective. Do NOT announce the absence of a link - phrases like "In an unrelated matter", "Separately", or "On a different note" read as stitched-together annotation rather than prose and will be marked down.
5. For a resolved Concession, assert the resolver atom after both endpoints and present it as settling the tension.
6. Write it as a continuous expository answer to the QUESTION - an argument a domain expert would publish, organized into 3-5 paragraphs, in which the planned relations are simply how the argument hangs together. Do NOT walk the plan relation by relation: a reader must not be able to tell that a plan exists. Restate a subject to carry an argument forward ("the radiocarbon date", "that attribution"), never the earlier sentence as a step ("the finding just reported", "the provision just stated", "the ordering just described"). Use precise and formal language, avoiding vague generalizations and rhetorical fillers. Aim for 550-650 words; below 500 is too short.
7. Do NOT mention the relation plan, the senses, the strength bands, the validity labels, or the fact that some content is incorrect. Do NOT hedge with "assume", "might", "possibly", "allegedly", "supposedly", "reportedly", or "it is claimed" around planned-invalid relations - assert them in the same register as the valid ones. Do NOT add headings, bullet

```
   lists, or annotations.
8. Wrap the RESPONSE in a code block.

Your Task:
QUESTION: [QUESTION_PLACEHOLDER]
RELATION PLAN: [PLAN_PLACEHOLDER]
RESPONSE:
```

Instruction 3's per-sense vocabulary is what makes the gold graph recoverable; the explicit exhaustiveness requirement for Alternative and Disjunction is what makes the CONTRADICTION/EXCLUSIVE/CO_NECESSITY distinction present *in the text* rather than only in the annotation.

Coordination is not a coverage failure. A coupling defined over the *joint* truth values of two atoms cannot be realized without a construction that scopes over both: "either X or Y" for Alternative, "at least one of X or Y" for Disjunction, "although X, Y" for Concession. Instruction 3 therefore *requires* two atoms to share a clause complex, and since Alternative, Disjunction, Restatement and Concession are exactly the senses instruction 5 of P3 makes mandatory, every family contains at least one such pair by construction. An earlier wording of instruction 2 forbade merging "two atoms into one clause" and V4 defined `merged` as content "fused into a clause that also asserts another atom", which made the two instructions unsatisfiable together: on the first live runs, 12 of 14 non-`asserted` flags fell on an endpoint of one of these senses, and generators were rejected for obeying the prompt. Both now turn on *separability* rather than clause count: an atom is `merged` only when it can no longer be negated on its own, and two atoms joined by a connective are both `asserted`. The 100% coverage bar is unchanged. Instruction 7 is the anti-leakage clause, verified by V3 — but only after both were brought into agreement, because V3's definition was for a time *broader* than instruction 7 and reproduced the same contradiction one gate over. Instruction 7 names the plan, the senses, the strength bands and the validity labels; V3 additionally flagged "any span ... referring to *claims*, *atoms*, *statements*" and "any label, tag, index", which catches both the coordinations instruction 3 mandates and ordinary scholarly usage of "claim" or "outcome" about the subject matter. On the first live runs this rejected every Claude family with 16 and 9 leakage spans, all of them instruction-3 compliance. V3's leakage clause now reads narrowly and exempts those constructions explicitly, exactly as V4's `merged` was narrowed above. Its second half matters more than it looks: if the generator hedges around planned-invalid relations, a miner can detect invalidity from register alone and the benchmark measures hedge-detection instead of reasoning — LoReFact guards the same risk in their Table 13 instruction 5. That half also has to name the same *words* V3 checks: it previously omitted "possibly" and "it is claimed", and a generator following instruction 3's suggested Disjunction wording drifted from "one or both" to "and possibly both" — graded against a list it had never been given. The two words were added; the *scope* was not. Widening the clause from "around planned-invalid relations" to "anywhere in the response" is the obvious next step and it is a trap: it collapses output length. Measured on one family, with everything else held fixed and the result deterministic across repeats — 581 words under the original wording, 637 with the word list widened, 308 once "ANYWHERE" was added, and 144 once a further sentence prohibited a specific phrasing. Each additional prohibition bought a shorter answer until the response fell below instruction 6's 500-word floor and P4 failed outright. The residual mismatch is therefore deliberate: V3 may flag a hedge that instruction 7 did not strictly forbid, and that is preferable to a prompt whose prohibitions suppress the prose they are meant to discipline.

A.5 P5 — parameterized perturbation

P5 — parameterized perturbation

Inputs: `[RESPONSE_PLACEHOLDER]`, `[PLAN_PLACEHOLDER]`, and `[OPERATOR_PLACEHOLDER]` — filled with *one* of the ten call expressions of instruction 3, e.g. `wrong_sense(r003, "Contrast")` or `shuffle_order(adjacent)`. An `edge` argument is a `relations[].id` from the plan.
Output: Two code blocks: the perturbed response, then the JSON diff.
Gate: Exactly one operator applied; length within 15% (except the three operators that add or remove a sentence); all other relations preserved verbatim.

One prompt covering all ten operators, so ladders are reproducible and the diff is machine readable.

```
Instructions:
1. You are given a RESPONSE, its RELATION PLAN, and one PERTURBATION to apply.
2. Apply ONLY the named perturbation. Change the minimum number of words
   needed. Every other relation, atom, and sentence must be preserved verbatim.
3. The perturbations are defined as follows.
   - wrong_sense(edge, new_sense): rewrite the connective realizing that edge so
     the text draws new_sense instead. Keep both atoms' content unchanged.
   - direction_reversal(edge): rewrite so the relation runs from the target to
     the source. Keep both atoms' content unchanged.
   - exhaustiveness_flip(edge): if the edge is Alternative, rewrite so the two
     claims merely oppose without being exhaustive (both could be false). If it
     is Contrast, rewrite so exactly one must hold.
   - remove_resolution(edge): delete the resolver sentence and any reference to
     it, leaving the conceded tension unresolved. Change nothing else.
   - inject_contradiction(atom): add ONE new sentence asserting a claim that
     directly contradicts the named atom. Place it at least two sentences later.
   - break_chain(edge): remove or negate the link realizing that edge so the
     causal chain no longer connects, keeping both endpoint atoms present.
   - shuffle_order(level): reorder the response's sentences without changing any
     wording. level=full permutes all sentences; level=block permutes paragraph
     blocks; level=adjacent swaps one adjacent pair.
   - spurious_relation(atom_a, atom_b): add a connective asserting a causal or
     contradictory link between two atoms the original response kept separate.
     Add no new factual content.
   - drop_relation(edge): remove the connective realizing that edge so the two
     atoms are merely adjacent assertions. Keep both atoms.
   - ordering_only(edge): rewrite a Precedence edge as Succession, or the
     reverse, preserving the actual chronology and adding NO causal or
     inferential language. The truth of neither atom may become dependent on
     the other.
4. Do not change the response's length by more than 15 percent, except for
   inject_contradiction (which adds one sentence) and remove_resolution /
   break_chain (which remove one).
5. Output the perturbed RESPONSE in a code block, then a second code block
   containing a JSON diff:
   {"operator": "...", "target": ..., "sentences_changed": [...],
    "relations_added": [...], "relations_removed": [...],
    "relations_relabeled": [...]}

Your Task:
RESPONSE: [RESPONSE_PLACEHOLDER]
```

```
RELATION PLAN: [PLAN_PLACEHOLDER]
PERTURBATION: [OPERATOR_PLACEHOLDER]
```

The `ordering_only` operator's final sentence is the whole content of the invariance test of Section 4.3: if the rewrite introduces any inferential language, the edit is no longer factor-neutral and the test is void.

A.6 V1 — round-trip relation recovery

V1 — round-trip relation recovery (the load-bearing gate)
Inputs: `[RESPONSE_PLACEHOLDER]`, `[ATOMS_PLACEHOLDER]` — a mapping keyed $1..N$ by atom position, so endpoints are nameable without the plan. **The plan is deliberately withheld.**
Output: A JSON list of recovered relations: source, target, sense, coupling, strength band, resolution.
Gate: Admission: a majority of the committee must recover $\geq$80% of planned relations with the correct coupling and $\geq$70% with the correct sense. Unrecovered edges are dropped or the item regenerated — never kept as gold.

The acceptance gate. An independent model, shown the response and its atoms but *not* the plan, re-derives the relation graph; an item enters the benchmark only if its planned graph is actually recoverable.

```
Let's define a function named recover_relations(response: str,
atoms: dict[str, str]).
Given a RESPONSE and the ATOMS drawn from it, the function returns the
list of logical relations the response ITSELF draws between those atoms.

ATOMS maps an atom's NUMBER to its text, numbered from 1. Use those numbers, as
integers, to identify atoms in your output: if the relation holds between the
atoms numbered "3" and "7", emit source 3 and target 7. Never renumber them and
never count from zero.

For each ORDERED pair (i, j) that the response actually relates, emit one object:
  {"source": i, "target": j,
   "sense": one of ["Cause-Effect","Effect-Cause","Evidence","Condition",
     "Restatement","Instantiation","Contrast","Concession","Alternative",
     "Disjunction","Precedence","Succession"],
   "coupling": one of ["entailment","contradiction","equivalence","exclusive",
     "co_necessity","none"],
   "strength_band": one of ["strong","moderate","weak"],
   "resolved": true or false or null}

Rules:
- Emit a relation ONLY if the response itself draws that connection, as written
  or as a clear step in the author's argument. Do not emit a relation that is
  merely plausible in general.
- Use "Alternative" only when the two claims cannot both be true AND cannot both
  be false. Use "Contrast" when they merely cannot both be true.
- Use "Disjunction" only when the response rules out both claims being false.
- Use "Precedence" or "Succession" only when the ordering carries no truth
  dependence; set their coupling to "none".
- "coupling" is not an independent choice: it must be the coupling that the
  "sense" you chose compiles to. Cause-Effect, Effect-Cause, Evidence,
  Condition and Instantiation are "entailment"; Restatement is "equivalence";
```

```
  Contrast and Concession are "contradiction"; Alternative is "exclusive";
  Disjunction is "co_necessity"; Precedence and Succession are "none".
- Pairs the response leaves unrelated must be omitted entirely.

The function returns a JSON list. Note that your response will be passed to the
python interpreter, SO NO OTHER WORDS!

recover_relations(response="[RESPONSE_PLACEHOLDER]", atoms=[ATOMS_PLACEHOLDER])
```

The gate. An item is admitted only if a majority of the V1 committee recovers $\geq 80\%$ of its planned relations with the correct *coupling* and $\geq 70\%$ with the correct *sense*. Unrecovered edges are dropped from gold or the item is regenerated — **never** retained as unrecoverable gold, which would penalize a correct miner for failing to read something that is not in the text.

Coupling recall is computed by compiling the recovered sense. Both rates are read off the recovered *sense*: coupling recall compares COMPILE[recovered] against COMPILE[planned] rather than reading the model's self-reported `coupling` field. COMPILE is the authority for that mapping everywhere else — the builder asserts gold cannot contradict it (Section 7) and the plan parser rejects a relation whose named coupling disagrees with its sense — so a free-choice label cannot outvote the sense it is derived from. This is also what makes coupling the *coarser* judgment its higher threshold assumes: a model that answers Evidence where the plan said Cause-Effect misses the sense but still earns coupling credit, since both compile to ENTAILMENT. That asymmetry is why $0.80 > 0.70$ here, why the reported recalls run 0.88 against 0.75, and what licenses grading lossily migrated seed edges on coupling alone (Section 5.5). Grading the self-reported field instead makes a *perfect* recovery fail whenever the model labels its coupling non-canonically — returning `entailment` beside a correctly recovered Concession scores sense 1.00 and coupling 0.00.

A.7 V2 — exhaustiveness adjudication

V2 — exhaustiveness adjudication
Inputs: `[CLAIM_A_PLACEHOLDER]`, `[CLAIM_B_PLACEHOLDER]`, `[RESPONSE_PLACEHOLDER]`. Run on V1s conflict edges.
Output: One letter: A exhaustive, B non-exhaustive, C disjunction, D neither.
Gate: Unanimity of the committee is required to assign an EXCLUSIVE gold label; a split vote ships the edge `low_agreement` (Section 9, R1).

The Contrast/Alternative boundary is the design's hardest label (Section 3.4). This prompt makes it a *procedure* rather than a matter of annotator taste.

```
Let's define a function named adjudicate_exhaustiveness(claim_a: str,
claim_b: str, response: str).

Given two claims and the response that asserts both, the function decides the
logical relationship between their truth values, returning exactly one of:
  "A" - EXHAUSTIVE EXCLUSION: exactly one of A and B holds. They cannot both be
        true, and they cannot both be false.
  "B" - NON-EXHAUSTIVE OPPOSITION: they cannot both be true, but both could be
        false (some third possibility exists).
  "C" - DISJUNCTION: at least one holds, and both may hold together.
  "D" - NEITHER: the truth values are not coupled in any of these ways.
```

```
To decide between A and B, ask: is there a coherent third state of the world in
which A and B are BOTH false? If yes, answer B. If no such third state exists,
answer A.
To decide between A and C, ask: could A and B both be true at once? If yes,
answer C.

Answer with the single letter only. Note that your response will be passed to
the python interpreter, SO NO OTHER WORDS!

adjudicate_exhaustiveness(claim_a="[CLAIM_A_PLACEHOLDER]",
                          claim_b="[CLAIM_B_PLACEHOLDER]",
                          response="[RESPONSE_PLACEHOLDER]")
```

The "coherent third state" test is the operative decision procedure, and it maps directly onto the factor tables: a third state where both are false is exactly the $(0, 0)$ world that EXCLUSIVE penalizes and CONTRADICTION permits.

A.8 V3 — naturalness and leakage audit

V3 — naturalness and leakage audit
Inputs: `[RESPONSE_PLACEHOLDER]`.
Output: JSON: `fluency`/`formality`/`organization` (1–5), plus `leakage`, `hedging`, `artifacts` span lists.
Gate: All three scores ≥ 4, and `leakage` and `hedging` both empty.

```
Let's define a function named check_response(response: str).
The function audits a generated response and returns a JSON object:
  {"fluency": 1-5, "formality": 1-5, "organization": 1-5,
   "leakage": [list of quoted spans], "hedging": [list of quoted spans],
   "artifacts": [list of quoted spans]}

- fluency, formality, organization: 5 = indistinguishable from expert human
  prose; 1 = badly degraded.
- leakage: any span revealing the response was generated from an annotation
  plan. Read this NARROWLY: naming a relation type or sense; naming the plan,
  the strength bands, or the validity labels; referring to "claims", "atoms" or
  "statements" as the objects the text was assembled from; or any label, tag,
  numeric index, or bracketed marker. Do NOT flag a connective or coordination
  that merely realizes a relation - "either X or Y", "at least one of X or Y",
  "the two cannot both be true, and one of them must hold", "although X, Y",
  "before", "after", or an explicit statement that an ordering carries no
  causal or inferential force. A writer is REQUIRED to use those, and a
  coupling over two claims' joint truth cannot be expressed without one. Do NOT
  flag ordinary scholarly vocabulary used about the subject matter, such as
  "claim", "outcome", "reading", "account" or "finding".
- hedging: any span using "assume", "might", "possibly", "allegedly",
  "supposedly", "reportedly", or "it is claimed", which would signal to a
  reader which content is unreliable.
- artifacts: enumerated list structure, headings, repeated template phrasing, or
  a sentence that reads as a stitched-together claim rather than prose.

Return the JSON object only. Note that your response will be passed to the
python interpreter, SO NO OTHER WORDS!
```

```
check_response(response="[RESPONSE_PLACEHOLDER]")
```

Gate: all three scores ≥ 4, and `leakage` and `hedging` both empty.

Leakage is read narrowly, and the spans are recorded. A coupling over two claims' joint truth cannot be realized without a construction that scopes over both (Section A.4), so the connectives P4 instruction 3 *mandates* are not evidence of a plan — they are evidence the writer complied. Leakage therefore means naming the apparatus (a sense, the plan, a band, a validity label, a tag or index), not using the discourse markers the apparatus requires; and "claim", "outcome" or "reading" used about the subject matter is ordinary scholarly English. `hedging` is deliberately *not* narrowed: a hedge is detectable from register alone and is the risk instruction 7 exists to prevent. Because a bare count is undiagnosable — "leakage: 16" reads as leaky prose when every span was a mandated coordination — the rejection record now carries a bounded sample of the span *text*. `artifacts` is recorded on the same footing but never gated: enumerated structure and template phrasing are quality signals rather than admission criteria.

The auditor is not the generator. V3 is a single-rater judgment, and Section 6.2 excludes the model that ran P3/P4 from an item's committee (R3, self-generation bias). That exclusion matters more for V3 than for any other validator, and it is enforced rather than assumed: the run resolves a per-generator auditor and V3 executes on it. Measured on the first live runs, the same deepseek response scored 5/5/5 with zero leakage under its own auditor and 3/4/4 with six leakage spans under a Claude auditor. The strict auditor was the more accurate one — it alone caught a genuine "possibly" hedge — so the divergence is not noise to be averaged away. The effect survived the leakage narrowing and is the reason the exclusion is now wired: on identical prose with the identical narrowed prompt, a self-auditing deepseek flagged five coordinations the prompt names *verbatim* as exempt, where a distinct auditor flagged one real span naming a sense. A weak self-auditor does not merely add noise — it rejects items for compliance.

Two details are load-bearing. Eligibility is decided by `model_id`, not by name: a committee may legitimately list one model under several labels for the agreement statistics, and a by-name check then returns the generator and the self-audit persists silently — which is exactly what the Claude committee's first entry did. And when no distinct auditor can be resolved or built, the run *warns and degrades* rather than aborting, because a self-audit is a weaker result and not an invalid one; trading a whole corpus for a provenance caveat would be the wrong exchange.

V3 votes; one rater does not decide. Excluding the generator is necessary but not sufficient, because V3's judgments diverge across models far more than the other validators' do. Measured on one response, identical prose and identical prompt: OPUS-5 found 0 leakage spans, SONNET-4.6 found 5, OPUS-4.8 0, OPUS-4.7 0. Resolving a single auditor therefore made admission depend on *committee ordering* — and the model that happened to sit first was the lone outlier, which rejected the family. V3 now runs on every eligible auditor and a facet rejects only on a strict majority (`v3_rule`), matching the rules already declared for V1 (majority) and V2 (unanimity).

Each facet votes *separately* rather than voting on the verdict as a whole. On the panel above the same four auditors agreed unanimously on one hedging span while splitting 1–3 on leakage; pooling the two would have let the agreed hedge carry the disputed leakage into the rejection reason and hidden the disagreement. Every auditor's vote is recorded, so a lone dissenter stays visible — distinguishing "every model saw it" from "one model did" is the point of voting, and it is also the evidence needed to decide later whether a facet belongs in the gate at all. Voting additionally

absorbs the boundary instability seen in repeat audits of a single family, whose `organization` score came back 3, 3, 4 against a floor of 4.

A.9 V4 — atom coverage and realized positions

V4 — atom coverage and realized positions
Inputs: `[RESPONSE_PLACEHOLDER]`, `[ATOMS_PLACEHOLDER]`.
Output: Per atom: status (`asserted`/`altered`/`missing`/`merged`), span, and sentence `position`.
Gate: 100% `asserted`. Two atoms joined by a connective are *both* asserted — `merged` is reserved for genuine loss of content (Section A.4 note). The `position` output re-verifies P3s window constraint against the realized text (Section 9, R2).

```
Let's define a function named check_atom_coverage(response: str,
atoms: list[str]).
For each atom, the function decides whether the response asserts it, returning a
JSON list of objects:
  {"index": i,
   "status": one of ["asserted", "altered", "missing", "merged"],
   "span": the quoted span of the response that asserts it, or null,
   "position": the 1-based index of that span among the response's sentences}

- "asserted": the response asserts the atom's full content, in any wording.
- "altered": the response asserts something related but changes the atom's
  content (a different number, date, entity, or polarity).
- "missing": the response does not assert the atom.
- "merged": the atom's content has been absorbed into another atom's, so it is no
  longer separately assertable - you could not negate this atom alone and leave
  the other standing. Reserve this for genuine loss of content.
  Two atoms joined by a connective are NOT merged: "either X or Y", "at least one
  of X or Y", "although X, Y", "X, whereas Y", and "X; that is, Y" each assert
  BOTH atoms, because each remains independently true or false. Mark both
  "asserted", with the span covering the part that asserts each one.

Return the JSON list only. Note that your response will be passed to the python
interpreter, SO NO OTHER WORDS!

check_atom_coverage(response="[RESPONSE_PLACEHOLDER]", atoms=[ATOMS_PLACEHOLDER])
```

Gate: 100% `asserted`. The `position` field has a second job — P3 constrains *planned* positions, but P4's realized sentence order may drift, so the window constraint must be re-verified against realized positions post hoc (Section 9, R2).